SOLARINET · DEVELOPER MANUAL

SolariNet Developer Manual

How SolariNet is built, where each piece runs, how the pieces talk to each other and what to be careful of when you change them.

Edition	2026-11-03 · first edition
Source	`main` at `a28bdf2`; live release `rc-20261103-48bd836-io` on xenon
Audience	Engineers and agents changing SolariNet code, schema or deployment
Supersedes	`docs/SolariNet_Developer_Reference.html`
Owner	Jason C. Kay

CONTENTS

Contents

Chapter 1 · About this manual

- Who this is for
- Observed, inferred and unverified
- Conventions

Chapter 2 · System context

- What SolariNet is
- Technology stack
- House rules that shape the code

Chapter 3 · Network, DNS, PKI, identity and messaging

- Addressing and segments
- Named hosts
- DNS
- Certificates and the two CAs
- Identity
- Messaging

Chapter 4 · Deployment by host

- What runs where
- xenon systemd units
- Other hosts

Chapter 5 · The C tier

- Overview
- The shared library
- SCP: the wire protocol
- Transport and delivery
- solariServer internals
- The ctl socket
- Enrollment
- Agents and workers
- Build options

Chapter 6 · Dashboard

- Shape of the dashboard
- Request lifecycle
- Sessions and sign-in
- Policy and CSRF
- Databases and the ctl socket
- The front end without a build step

Chapter 7 · Data

- Two databases
- Migrations and the ledger
- Provenance
- Types and conventions
- Triggers
- Domain models

Chapter 8 · SoR sync and messaging

- Why change data capture
- Outbox and emitter
- Appliers
- dns-release
- Inbound writers

Chapter 9 · Alerting and notification

- From probe to phone
- The dead-man's switch
- Senders

Chapter 10 · Provisioning

- From bare metal to a running stack
- Network boot
- First boot and enrollment
- STACK_APPLY and Ansible

Chapter 11 · Inventory I1

Scope of Inventory I1
Data model
Items, software and licenses
Routes

Chapter 12 · RackWire and the status panel

RackWire
Status panel

Chapter 13 · Security boundaries

Trust zones
Rules that hold the boundaries
Known gaps

Chapter 14 · Build, test and CI

Building and testing locally
Continuous integration

Chapter 15 · Release and deploy

From commit to xenon
Versions and order of operations

Chapter 16 · Repository map, standards and cautions

Repository map
Coding standards
Operational cautions

Chapter 17 · Interrelationship matrix

Component matrix
Connections in detail

Appendix A · Ports

Ports

Appendix B · Discrepancy register

Discrepancy register

Appendix C · Glossary

Glossary

CHAPTER 1

About this manual

Who this is for

This manual is for anyone who changes SolariNet: its C daemons, its PHP dashboard, its two databases, its sync and alert daemons, its provisioning tooling or the way it is released onto xenon. It explains how the system is put together, how each part talks to the others, and where the sharp edges are. It is not an operator's guide; the User Manual covers day-to-day use, and the API Expansion Manual covers adding routes and integrations.

It replaces `docs/SolariNet_Developer_Reference.html`. Every claim here was checked against the repository at `main a28bdf2` or against the running system on 2026-11-03, rather than copied from older documents. Where an older document disagrees with what is true now, Appendix B records the disagreement.

Observed, inferred and unverified

TAG	MEANING
OBSERVED	Read from the live system on 2026-11-03: `systemctl`, `ss`, `dig`, Apache configuration, or a read-only `SELECT` against a database.
INFERRED	Reasoned from code or configuration but not seen running. Treat it as likely, not certain.
UNVERIFIED	Stated by a document or a deploy tree, but neither the code path nor the live state was checked.

Untagged statements about code come from the source at `a28bdf2`, with the file named. Hostnames, addresses and paths are given in full; credentials never are. For a secret, the manual names only the file that holds it.

Conventions

- **SoR** means the system-of-record database `sor` on cesium. **Monitoring DB** means the `solarinet` database on xenon. They are separate servers with separate schemas and separate migration trees.
- Relative paths (`src/`, `dashboard/`, `deploy/`…) are repository paths; absolute paths (`/opt`, `/etc`, `/run`, `/srv`) are host paths.
- Ports are written `host:port` or `:port`. All internal names are in `akoria.net` unless written otherwise; Active Directory is `akoria.org`.
- Passages on SoR migration 037 behavior are marked in the source for checking against the merge; they describe it as current.

Diagram legend

Teal outlines are built and run by SolariNet; grey outlines are infrastructure it depends on; dashed outlines are external, planned, offline or stubbed. Plain arrows are requests or SQL, dashed teal arrows asynchronous messages, violet arrows ctl-socket calls and pink arrows alert or failure paths. A pink dashed frame is a trust zone. In entity–relationship diagrams, **bold** rows are primary keys, blue rows foreign keys, and arrows point at the referenced table.

CHAPTER 2

System context

What SolariNet is

SolariNet is the monitoring, inventory and provisioning system for the SolariNet homelab, a self-hosted network of about 40 named hosts across eight logical networks. It does four jobs:

- **Monitor.** Agents on enrolled hosts report health; monitor agents probe services; a server ingests both, raises alerts and keeps history.
- **Record.** A system-of-record database (the SoR) holds the intended state of the network: entities, addresses, interfaces, DNS, functions, inventory and provisioning catalogs, each row stamped with where it came from.
- **Apply.** Changes to the SoR are captured and pushed out to the systems that act on them: BIND zones and Samba Active Directory.
- **Provision.** Bare-metal hosts are network-booted, installed, enrolled and brought to a software stack with Ansible.

The operator works through one web dashboard, used mostly on an iPad. An LED status panel on lithium shows the same state at a glance.

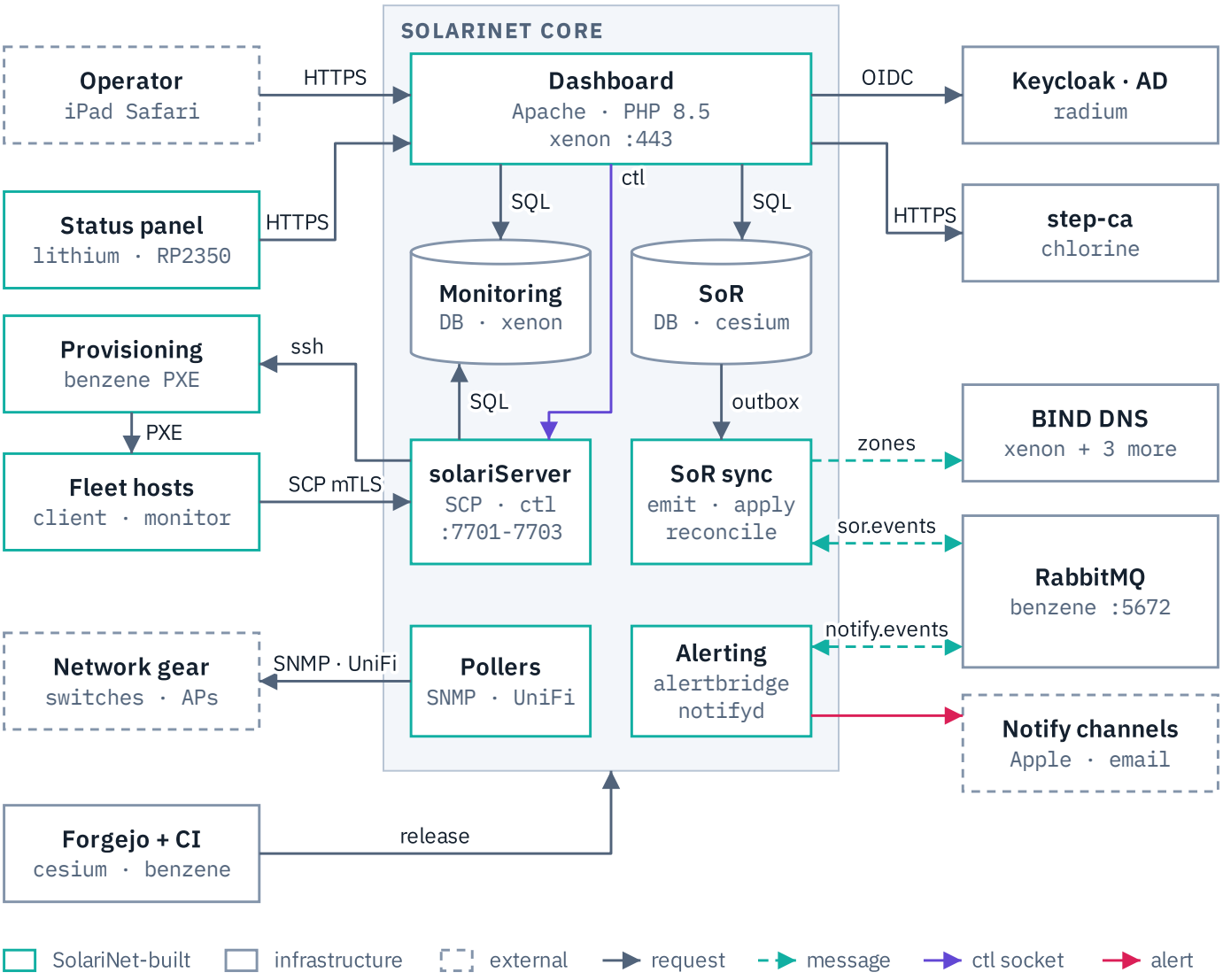


Figure 1 System context. SolariNet-built components sit in the center; infrastructure and people around them.

Technology stack

LAYER	TECHNOLOGY	NOTES
Daemons	Standard C (C99), CMake + Make	nng 1.x over mbedTLS for transport; SQLite for agent spools; cJSON. Linked statically in release builds.
Dashboard API	PHP 8.5 on php-fpm behind Apache 2	Functional style, no framework, no Composer runtime dependencies. One front controller, `dashboard/api/index.php`.
Dashboard UI	React 18.3.1 + Babel standalone 7.29.0, vendored	JSX compiled in the browser by a Web Worker. No bundler, no Node toolchain, no build step.
Sync, alerting, provisioning glue	Python 3 (system interpreter + per-release `.venv`)	`deploy/sorsync`, `deploy/alertbridge`, `deploy/notify`, `deploy/unifi`, `deploy/fleet`.
Databases	MariaDB 11.8.6 (monitoring, xenon) · MariaDB 12.3.2 (SoR, cesium)	SoR replicates to benzene. IP columns use the native `INET6` type.

LAYER	TECHNOLOGY	NOTES
Messaging	RabbitMQ (AMQP 0-9-1) · Mosquitto (MQTT)	Both on benzene. RabbitMQ carries SoR change events and notifications.
Identity	Samba AD `AKORIA.ORG` · Keycloak 26.6.4 (OIDC)	On radium. The dashboard is an OIDC client; local accounts are break-glass.
PKI	SolariNet internal CA in solariServer · step-ca on chlorine	The first issues agent mTLS certificates; the second issues host and service certificates.
Panel firmware	C, pico-sdk 2.1.1, RP2350	Galactic Unicorn 53×11 LED matrix.

House rules that shape the code

These come from the project instructions and the operator standard, and they explain choices that look odd from outside:

- **The application is standard C.** Every daemon on the monitoring path is C with Makefiles and CMake. Python is used for glue that is not on the monitoring path.
- **No server-side JavaScript, ever, and no build step where none exists.** The dashboard UI ships its JSX source; the browser compiles it.
- **camelCase** for variables, functions and structures in C and PHP alike, even where the language community uses snake_case. Database columns stay snake_case.
- **Prefer functions to objects.** The C core uses opaque handles; PHP classes are mostly static namespaces (`Auth`, `Policy`, `Sor`).
- **Comment intent.** Each program, function and non-obvious control structure states its purpose, input and output.
- **A socket accepting a connection is not health.** Learned from the 2026-07-07 cesium incident, where Forgejo returned errors for about 26 hours while a bare TCP check stayed green. Health checks must exercise the application.

CHAPTER 3

Network, DNS, PKI, identity and messaging

Addressing and segments

The SoR defines eight logical networks. Every subnet is a /24, and the second octet names the network: `10.0` core, `10.1` production, `10.2` IoT, `10.3` peripherals, `10.4` personal, `10.5` workstations, `10.6` network management and `10.7` security. OBSERVED

NETWORK	SUBNETS	PURPOSE (SOR)
core	10.0.0.0/24, 10.0.1.0/24	Core networking and grandfathered core services
production	10.1.0.0/24	Production services (directory, SoR, git)
IoT	10.2.0, 10.2.2, 10.2.3, 10.2.4	Smart-home and IoT devices
peripherals	10.3.0.0/24	Printers, radios and other peripherals
personal	10.4.1, 10.4.4	Personal devices
workstations	10.5.0, 10.5.2, 10.5.87	Workstations and SBCs
netmgmt	10.6.0, 10.6.6	Network management, timing, Grafana
security	10.7.0.0/24	Security, cameras, NVR, the internal CA

All eight networks are enabled; none is flagged as the management network and none is yet eligible for the address allocator. The `vlans` table is empty. xenon's own interface is `10.0.0.20/8`, so from xenon every segment is directly on-link. OBSERVED The segments are therefore an addressing convention on a flat routed fabric, not enforced layer-2 isolation. INFERRED Nothing in this manual should be read as a firewall boundary between segments.

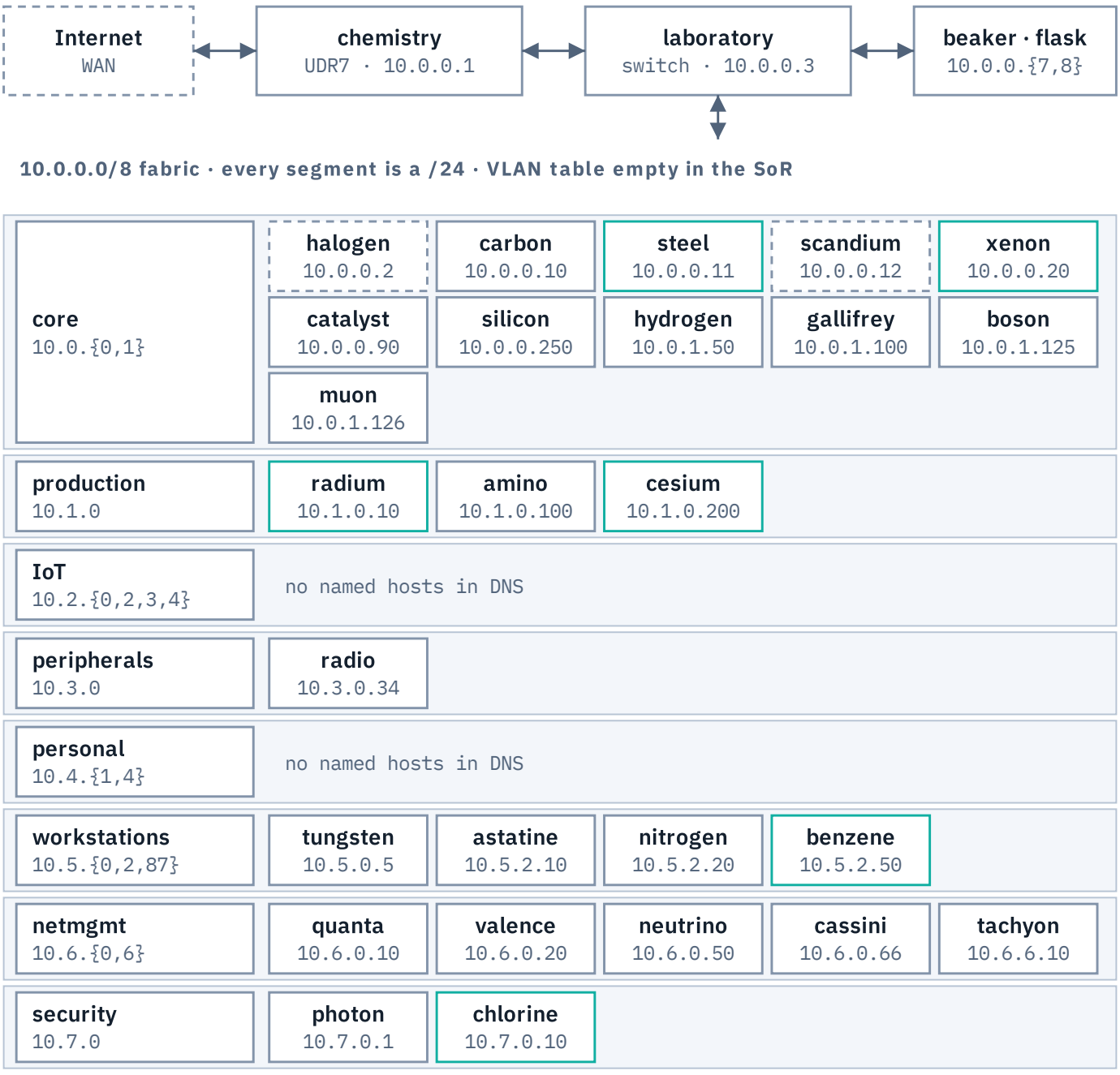


Figure 2 Logical network layout from the SoR `networks`/`subnets` tables and the `akoria.net` zone, with every named host placed in its /24.

Named hosts

HOST	ADDRESS	ROLE
chemistry	10.0.0.1	UniFi Dream Router 7: gateway, firewall, UniFi controller, DHCP
halogen	10.0.0.2	Offline
laboratory	10.0.0.3	Core switch

HOST	ADDRESS	ROLE
beaker, flask	10.0.0.7, .8	Wi-Fi 7 access points
carbon	10.0.0.10	NAS
steel	10.0.0.11	ZimaBlade, openSUSE Tumbleweed; BIND secondary
scandium	10.0.0.12	Planned
xenon	10.0.0.20	Ubuntu 26.04. solariServer, dashboard, monitoring DB, sync and alert daemons, BIND primary
catalyst	10.0.0.90	MoCA bridge
silicon	10.0.0.250	Caddy reverse proxy
hydrogen	10.0.1.50	Workstation
gallifrey	10.0.1.100	Second NAS
boson, muon	10.0.1.125, .126	KVM consoles
radium	10.1.0.10	Raspberry Pi CM5, Debian 13. Samba AD DC, Keycloak, BIND (AD zone)
amino	10.1.0.100	n8n
cesium	10.1.0.200	Dell R410, openSUSE Tumbleweed. SoR MariaDB, Forgejo
radio	10.3.0.34	Peripheral
tungsten · astatine · nitrogen	10.5.0.5 · 10.5.2.10 · 10.5.2.20	Workstations
benzene	10.5.2.50	openSUSE Tumbleweed. RabbitMQ, Mosquitto, netboot, Ansible, SoR replica, CI runner VM
quanta	10.6.0.10	Grafana
valence	10.6.0.20	In the hosts file only; role not established UNVERIFIED
neutrino · cassini	10.6.0.50 · 10.6.0.66	Network management
tachyon	10.6.6.10	GPS / PTP time source
photon	10.7.0.1	NVR
chlorine	10.7.0.10	Ubuntu 22.04. step-ca, BIND secondary
lithium	not in DNS or the SoR	Status-panel host (solariPanel)

DNS

`akoria.net` is authoritative on BIND. xenon is the primary; radium, chlorine and steel are secondaries. The SOA observed on all four was `xenon.akoria.net. hostmaster.akoria.net. 2026110302 3600 600 604800 300`. OBSERVED The NS set published in the zone is xenon, radium and chlorine; steel transfers the zone but is not listed as NS.

Zones are generated, not hand-edited. `netdb/gen-zones.py --source sor` renders forward and reverse zones from the SoR; the dns-release engine (`deploy/dns-release`) installs each render as a new generation, validates it, swaps it in, checks the served SOAs and keeps a journal so a bad generation is rolled back (Chapter 8). The

forward zone is transferred to 10.1.0.10, 10.7.0.10 and 10.0.0.11; reverse zones to 10.7.0.10 and 10.0.0.11. The live transfer lists are in `/etc/solarinet/dns-release.json`. `solari-dns-secondary-refresh` (a timer and a path unit on xenon) refreshes chlorine's secondary zone list when the release changes.

Service names are descriptive CNAMEs: `dashboard` → xenon, `sso` → radium, `git` → cesium, `db` → benzene, `ca` → chlorine, `ns1` → xenon. Each service is reached by name on :443 with SNI rather than on an arbitrary port.

xenon also serves a response-policy zone built daily at 04:30 from the StevenBlack block list (`solarinet-rpz-refresh.timer`). `akoria.org` is the Active Directory DNS zone, served by Samba on radium.

Certificates and the two CAs

SolariNet uses two certificate authorities for different things. Mixing them up is a common mistake.

CA	WHERE	ISSUES	HOW
SolariNet mTLS CA	Inside solariServer on xenon; public cert `run/pki/ca.pem`	Agent identities: CN `server`, `client.<fqdn>`, `monitor.<fqdn>`	CSR through the enrollment flow (Chapter 5) or `solariCtlClient sign`. Revocation through a CRL.
step-ca (Akoria internal CA)	chlorine, `https://chlorine.akoria.net`	TLS server certificates for hosts and services; S/MIME; SSH certificates; the panel's Wi-Fi client certificates	ACME and the step CLI. Kept manual: no SolariNet code signs with it.

The dashboard's CA page (`dashboard/api/lib/Ca.php`) reads from both: step-ca health and roots over HTTPS (`STEPCA_URL`, `STEPCA_CA_FILE`), the SolariNet CA certificate from `SOLARINET_CA_PEM`, and the per-node certificate inventory from the monitoring `node` table. It is read-only and fails soft: PHP never signs anything.

The mTLS CA's revocation tooling lives in `deploy/crl`: generate a CRL every six hours (valid 24 hours), publish it every minute, and have peers activate it hourly. Durable reports carry a CRL-activation receipt TLV so the server can see which CRL each agent enforces. These timers are **not installed on xenon** today. OBSERVED

Identity

radium runs the Samba AD domain controller for `AKORIA.ORG` (Kerberos :88, LDAP :389, LDAPS :636, DNS :53) and Keycloak 26.6.4 on :8443 with realm `akoria`. Keycloak federates users from AD and is the dashboard's OpenID Connect provider at `https://sso.akoria.org:8443/realms/akoria`, client `solarinet-dashboard`. OBSERVED

Keycloak's LDAP federation binds to AD with a domain account whose password must not expire; an expired bind password took down every dashboard login on 2026-08-17. Treat the bind account as infrastructure.

Messaging

benzene runs RabbitMQ (AMQP :5672, management :15672, vhost `solari`) and Mosquitto (MQTT :1883). OBSERVED RabbitMQ carries two exchanges that SolariNet depends on:

EXCHANGE	TYPE	PUBLISHER	CONSUMERS	ROUTING KEY
`sor.events`	topic, durable	sor_emitd (xenon)	sor_apply_dnsd, sor_apply_ad	`sor.<table>.<action>`
`notify.events`	topic, durable	alertbridge (xenon)	notifyd	`notify.<severity>`

MQTT is used by the notify `mqtt` sender and by the optional Tab5 push-approval daemon (`deploy/authbroker`), which is not wired into the dashboard. Whether AMQP and MQTT run with TLS in production was not checked; the example configurations use plaintext. **UNVERIFIED**

CHAPTER 4

Deployment by host

What runs where



Figure 3 Component deployment by host, as observed on 2026-11-03 (xenon and cesium from live reads; other hosts from DNS, port probes and repo deploy trees).

xenon carries almost all of SolariNet. Every SolariNet unit there runs a binary or script from the immutable release tree `/opt/solarinet/current`, which on 2026-11-03 pointed to `/opt/solarinet/releases/rc-20261103-48bd836-io`. OBSERVED

xenon systemd units

UNIT	STATE	RUNS
`solarinet-server`	active	solariServer (nng :7701–7703, ctl socket, lease, CA)
`solarinet-client`	active	solariClient for xenon itself, with its watchdog sibling

UNIT	STATE	RUNS
solarinet-monitor	active	solariMonitor probing the service targets
solarinet-snmp.timer	active (60 s)	solariSnmpPoll one-shot runs
sor-emit	active	sor_emitd: SoR outbox → sor.events
sor-apply-dns	active	sor_apply_dnsd: sor.apply.dns → zone render → dns-release
sor-reconcile	active	sor_reconcile_discovery: discovery results → SoR
alertbridge	active	alertEvent → notify.events, maintenance windows, dead-man's switch
notify	active	notifyd: notify.events → channel senders
unifipolld	active	UniFi controller → SoR every 15 s
solari-prov-finish.timer	active	Finishes provisioning runs; queues STACK_APPLY
solari-dns-secondary-refresh	active (timer + path)	Refreshes chlorine's secondary zone list
solarinet-rpz-refresh.timer	active (daily 04:30)	Rebuilds the RPZ
solari-backup@xenon.timer	active	Host backup
sor-apply-pihole	disabled	Pi-hole applier (Pi-holes retired)
solari-mdns-inspect	disabled	mDNS inspection helper

Supporting services on xenon: apache2, php8.5-fpm (pool solarinet as www-solari), mariadb 11.8.6, named, avahi-daemon and winbind. No scan-worker unit is installed, so discovery scan jobs are not claimed by a separate solariScanWorker on xenon. OBSERVED

> **DEVIATION FROM THE DESIGN**
>
> The repository defines dedicated service identities (solari for the server, www-solari for php-fpm, group solari-ctl). On xenon only php-fpm uses its own identity; every SolariNet daemon unit runs as User=jason. Code that assumes the server runs as solari is untested in production.

Other hosts

- **cesium**: the SoR (MariaDB 12.3.2, database sor, :3306) and Forgejo (git.akoria.net, HTTP :3000, SSH :2222). The canonical git remote is Forgejo; GitHub is a publication mirror.
- **benzene**: RabbitMQ and Mosquitto; the netboot web root on nginx :8080 and TFTP iPXE loaders; the Ansible tree under /srv/solari-provision/ansible; a MariaDB replica of the SoR (db.akoria.net); and the ephemeral libvirt VM that runs CI jobs (network 192.168.123.0/24).
- **radium**: AD, Keycloak and BIND for akoria.org; a secondary for akoria.net. The AD applier (sor-apply-ad) is designed to run here as root; its live state was not inspected. UNVERIFIED
- **chlorine**: step-ca and a BIND secondary. **steel**: a BIND secondary.
- **lithium**: solariPanel and the Galactic Unicorn panel on USB. It has no DNS name or SoR row.
- **Fleet hosts**: solariClient with its watchdog sibling and SQLite spool; optionally solariMonitor. Configuration under /etc/solari/.

CHAPTER 5

The C tier

Overview

The C tier is the monitoring path: agents that report, a server that ingests, and helpers around them. It is built from one CMake tree with a shared library, `lib/solari`, and three program trees, `src/client`, `src/monitor` and `src/server`.

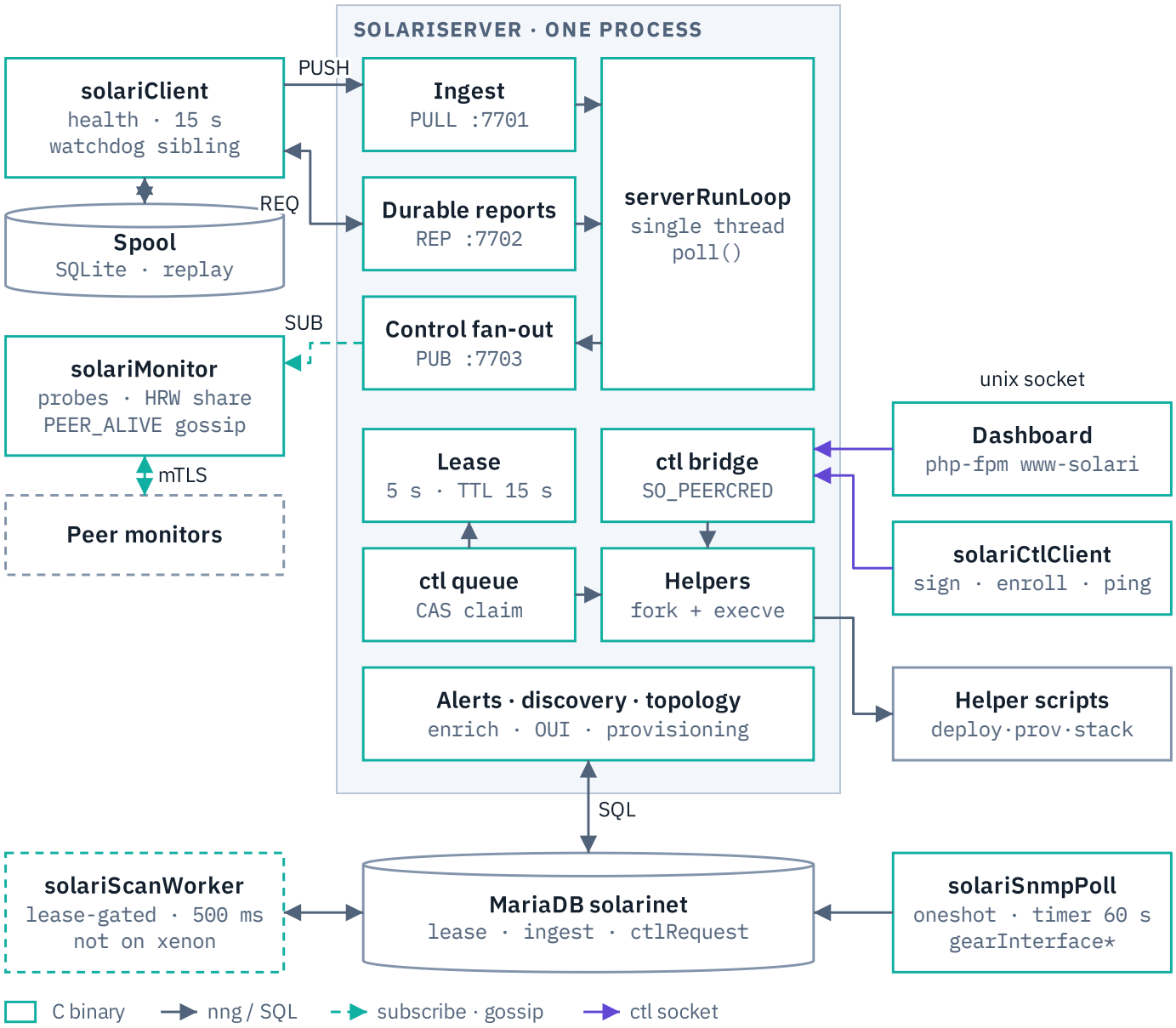


Figure 4 C tier: one solariServer process with its three nng listeners, the ctl bridge and the lease, and the binaries that talk to it.

BINARY	SOURCE	RUNS ON	ROLE
`solariServer`	`src/server/main.c` + `server*.c`	xenon	Ingest, durable reports, control fan-out, lease, ctl bridge, CA, alerting, discovery, topology, provisioning verbs
`solariClient`	`src/client`	every enrolled host	Host health every 15 s; executes control verbs; watchdog sibling
`solariMonitor`	`src/monitor`	xenon, benzene, outposts	Service probes shared across monitors by rendezvous hashing; peer gossip
`solariCtlClient`	`src/server/ctlClientMain.c`	xenon	Operator CLI for the ctl socket: `sign`, `ping`, `revoke-cert`, `enroll`
`solariScanWorker`	`src/server/scanWorkerMain.c`	not installed	Claims discovery scan jobs while the lease is held
`solariSnmpPoll`	`src/server/snmpPollMain.c`	xenon (timer)	SNMP interface and LLDP poller → `gearInterfaceCurrent/History`
`solariPanel`	`status-panel/daemon`	lithium	Dashboard ↔ LED panel bridge (Chapter 12); own Makefile, libcurl + OpenSSL

The shared library

MODULE	PURPOSE
`solariError`, `solariTime`, `solariLog`	Error codes, monotonic and wall time, levelled logging
`solariCrypto`	FNV-1a 64 (node IDs) and CRC-32 (frame trailer)
`solariFrame`, `solariTlv`, `solariMsg`	SCP framing, TLV encode/decode, message builders
`solariConfig`, `solariJson`	INI-style configuration; cJSON wrapper
`solariSpool`	SQLite spool for reports that could not be delivered
`solariNet`	nng sockets over mbedTLS with mutual authentication
`solariReporter`	Push-or-spool reporting shared by client and monitor

SCP: the wire protocol

Every message between agents and the server is one SCP frame. Integers are big-endian; the maximum frame is 1 MiB; the protocol version is 1.

```
u32 frameLen | 32-byte header | TLV payload | u32 crc32

header: magic "SN" | version | msgType | flags | reserved | tlvCount
        | sourceNodeId u64 | sendTimeUnixMs | seqNo | correlationId
```

CODE	MESSAGE	CODE	MESSAGE
0x01	HELLO	0x20	SURVEY
0x02	WELCOME	0x21	SURVEY_RESP
0x10	CLIENT_REPORT	0x30	CONTROL
0x11	MONITOR_REPORT	0x31	CONTROL_RESULT
0x12	WATCHDOG	0x32	DECOMMISSION
0x13	DISCOVERY_ADVERT	0x40	WHO_IS_PRIMARY
0x14	TOPOLOGY_REPORT	0x41	PRIMARY_IS
0x15	PEER_ALIVE	0x50 · 0x51	ACK · ERROR

Flags: `ACK_REQ` 0x01, `COMPRESSED` 0x02, `SPOOL_REPLAY` 0x04, `URGENT` 0x08. CONTROL frames carry one verb: `SET_CONFIG` 1, `DEPLOY` 2, `RESTART` 3, `PROBE_NOW` 4, `SET_SCHEDULE` 5, `DRAIN_SPOOL` 6, `PROVISION` 7, `DECOMMISSION` 8, `ADOPT_TARGET` 9.

A node ID is the FNV-1a 64-bit hash of `"fqdn|ROLE"`. The server checks that the certificate CN presented on the TLS session matches the claimed source node and its approved record. The installed nng 1.x API does not expose the peer's leaf certificate, so the server cannot yet compare the stored leaf fingerprint on live connections.

Transport and delivery

PORT	SERVER SOCKET	PEER SOCKET	CARRIES
:7701	PULL	PUSH	Best-effort frames: watchdog, discovery adverts, topology
:7702	REP	REQ	Durable reports (CLIENT_REPORT, MONITOR_REPORT, SURVEY_RESP), acknowledged
:7703	PUB	SUB	Control fan-out to clients and monitors

All three listen on `tls+tcp://0.0.0.0` with mutual TLS. **OBSERVED** `solariReporter` sends durable reports over REQ to the report-ack URL (derived from the primary URL by switching to :7702, with failover to other servers). It waits for an ACK correlated to the frame's sequence number and a 16-byte session TLV (0x0105), and it requires the server certificate's CN to be `server`. Durable reports also carry the CRL-activation receipt TLV. If no ACK arrives, the frame goes to the SQLite spool and is replayed later with `SPOOL_REPLAY` set. Delivery is at-least-once; the server de-duplicates by node, session and sequence.

solariServer internals

solariServer is a single-threaded `poll()` loop, `serverRunLoop` in `src/server/main.c`. It never blocks on one peer.

Lease: active and standby

Several servers can share one monitoring DB. Exactly one is active, decided by a lease row (`serverLease`) renewed every 5 s with a 15 s TTL in one conditional `UPDATE`. Only the active server drains ingest (in bursts of up to 64 frames), runs the ctl queue, and answers mutating ctl verbs. A standby answers only `PING`, `REQUEST_GET` and `DISCOVER_STATUS`.

Configuration

`server.conf` sections: `[identity]`, `[bind]`, `[tls]`, `[ca]`, `[lease]`, `[db]`, `[discovery]`, `[ingest]` and `[ctl]` (`enforcePeer`, `operatorUid`, `dashboardUid`, `socketGid`, `helperRoot`, `logDir`). The DB password is read only from the environment variable `SOLARI_DB_PASS`, set by the unit's environment file. Schema is created by migrations, never by the server: `--init-db` is retired.

Responsibilities

- **Ingest** writes reports to the monitoring DB and updates node, health and probe state.
- **Alerting** (`serverAlert.c`) evaluates rules, writes `alertEvent` rows with a disposition (suppress or publish) and PUBs them (Chapter 9).
- **Discovery and enrichment** (`serverDiscovery*.c`, `serverOui.c`, `serverScan.c`) manage scan jobs and annotate discovered hosts. The scan input is capped at about 4,096 hosts (a /20) per job.
- **Topology** (`serverTopology.c`) assembles LLDP and interface data from solariSnmpPoll.
- **CA** signs agent CSRs on approval and issues the enrollment certificate.
- **Provisioning verbs** (`serverProvision.c`) start helper programs for fleet install, image refresh, staging and stack apply.

The ctl socket

The ctl socket is the only way into solariServer from the same host. The dashboard uses it for every privileged action. It is an `AF_UNIX` stream socket at `/run/solari/solariCtl.sock`, observed as owner `jason`, group `solari-ctl`, mode 0660, inside `/run/solari` (mode 751). **OBSERVED**

```
request:  VERB key=value key=value\n        (≤ 16,384 bytes, one per connection, 2 s read
timeout)
reply:    OK [key=value …]  |  ERR <code> <message>   (≤ 32,768 bytes)
```

The server classes each peer by `SO_PEERCRED`: root or the configured `operatorUid` is OPERATOR, the `dashboardUid` is DASHBOARD, anyone else is UNKNOWN. Verbs fall into three classes:

CLASS	VERBS	WHO MAY CALL
Ordinary	PING, DISCOVER_STATUS, SURVEY, ADOPT, ASSET_SET, IGNORE, POOL_NEW, POOL_SET, RULE_SET, ALERT_ACK, CRIT_SET, ENROLL_SUBMIT, ENROLL_ACK	Dashboard or operator
Enqueue	REQUEST_SUBMIT, REQUEST_GET	Dashboard: wraps a privileged verb into a queued request

CLASS	VERBS	WHO MAY CALL
Privileged (default)	Everything else, including DECOMMISSION, RETIRE, SIGN, REVOKE_CERT, DEPLOY, FLEET_PROVISION, FLEET_IMAGE, IMAGE_REFRESH, PROV_STAGE, STACK_APPLY, DISCOVER, ENROLL_TOKEN, ASSET_REMOVE, TARGET_REMOVE, LIFECYCLE_SET, ASSET_PURGE, POOL_DEL, RULE_DEL	Operator peer only

Destructive verbs require a named operator (`op=`). `DECOMMISSION` is two-step: the first call replies `OK confirm=<token>` without changing anything; a second call echoing the token performs it. `docs/authz/AUTHZ_MANIFEST.json` is the authorization manifest: it lists the ctl verbs alongside the 194 PHP routes, and CI checks the code against it.

The ctl queue

The dashboard cannot call a privileged verb directly. It sends `REQUEST_SUBMIT`, which stores a row in `ctlRequest` with the requesting operator. On the active server, `serverCtlQueuePoll` claims rows by compare-and-swap, rebinds `op=` from the stored `requestedBy`, and replays the verb with operator authority. The dashboard polls with `REQUEST_GET`. This gives every privileged action an attributed, durable record.

Helpers

Long-running work is delegated to helper programs under `helperRoot`, started with `fork` + `execve` and no shell: `remote-deploy.sh`, `fleet-provision.sh`, `fleet-image.sh`, `image-refresh.py`, `prov-stage.py` and `stack-apply.py`.

Enrollment

A new agent gets its mTLS identity through a token exchange. Nothing private leaves the host.

STEP	AUTHORITY	EFFECT
Issue token	Operator; `ENROLL_TOKEN`	Creates a host- and role-bound row; stores only the SHA-256 digest of a 256-bit token and an expiry (default 900 s). The raw token is shown once.
Submit CSR	Bearer token; `POST /api/enrollments` → `ENROLL_SUBMIT`	The host generates its key and posts a CSR. C rechecks digest, expiry, host and role. Exact retries are idempotent by CSR fingerprint.
Approve	Operator, from the approval queue	One transaction signs the certificate and stores the approved node, leaf fingerprint and serial.
Pick up	Same token	Resubmitting returns the certificate and CA until acknowledged.
Acknowledge	Same token; `POST /api/enrollments/ack` → `ENROLL_ACK`	Records the acknowledgement and clears the token digest.
Reject	Operator	Marks the request rejected and clears the digest and CSR.

`deploy/enrollment/solari-enroll.sh` stages the returned files, verifies the leaf against the CA and checks the key pair before installing anything; `--resume` retries an acknowledgement without ever replacing identity material. `tests/integration/test_enrollment_e2e.py` covers the whole HTTPS → PHP → ctl → MariaDB → CA path in CI.

Agents and workers

solariClient

Collects host health every `sampleIntervalSec` (15), subscribes to control on :7703 and sends CLIENT_REPORT or spools it. The watchdog is the same binary: `--watchdog` starts a `--watchdog-of <pid>` sibling that reports every `watchdogIntervalSec` (5). Configuration is `/etc/solari/client.conf` with sections `[identity]`, `[schedule]`, `[server]`, `[control]`, `[tls]` and `[watch]`. Linux only.

solariMonitor

Probes tcp, udp, icmp, dns, ldap, http, mysql and amqp targets (up to 64 targets and 64 fleet members) every `roundIntervalSec` (30). Monitors share targets by highest-random-weight hashing with a replication factor, and gossip PEER_ALIVE to each other over mTLS PUSH/PULL; a peer's CN must be `monitor.<fqdn>` and match its node ID. Peers expire after `peerTtlSec` (90). `gossipIntervalSec` is parsed but not used.

solariScanWorker

A separate process holding `flock` on `/run/solari/solariScanWorker.lock`. While the lease is held it claims discovery scan jobs on a 500 ms loop. Unit templates exist in `deploy/systemd`, but none is installed on xenon.

solariSnmpPoll

Flags `--config`, `--community`, `--interval`, `--once`/`--loop` and `--lldp`. Shells out to `snmpbulkwalk` at run time and writes `gearInterfaceCurrent` and `gearInterfaceHistory`. On xenon it runs one-shot from `solarinet-snmp.timer` every 60 s.

Build options

CMAKE OPTION	DEFAULT	EFFECT
`SOLARI_BUILD_SERVER`	OFF	Build solariServer and its tools
`SOLARI_BUILD_CLIENT` · `_MONITOR`	ON	Build the agents
`SOLARI_BUILD_TESTS`	ON	Unity unit tests (30 `add_test` entries)
`SOLARI_WITH_IO`	OFF	nng + mbedTLS transport. Without it the agents build but cannot talk.
`SOLARI_WITH_SQLITE` · `_JSON`	OFF	Spool and cJSON support
`SOLARI_WITH_DISCOVERY_TOOLS` · `_SNMP`	OFF	Server-side discovery helpers; SNMP is called at run time
`SOLARI_BUILD_FUZZERS`	OFF	libFuzzer targets for the frame and TLV parsers
`SOLARI_VENDORED_DEPS` · `_ALLOW_SYSTEM_DEPS`	auto · OFF	Use `third_party/` sources or system packages

The language is C99 with `-Wall -Wextra -Wpedantic`; the default build type is Debug. The top-level `Makefile` wraps CMake with targets `all`/`core`, `spool`, `client`, `monitor`, `server`, `cross-arm64`, `cross-arm32`, `windows`, `test` and `clean`; toolchain files are in `cmake/toolchains/`. A full local server build needs `-DSOLARI_BUILD_SERVER=ON -DSOLARI_WITH_IO=ON -DSOLARI_WITH_SQLITE=ON -DSOLARI_WITH_JSON=ON -DSOLARI_WITH_DISCOVERY_TOOLS=ON`, and a clone with `--recursive` for the vendored dependencies.

CHAPTER 6

Dashboard

Shape of the dashboard

The dashboard is two programs that share a directory. `dashboard/api` is a PHP JSON API with one front controller; `dashboard/public` is a static single-page app that calls it. Apache serves both from the release tree.

ITEM	VALUE
Virtual host	`dashboard.akoria.net` on *:443; aliases `solarinet`, `monitor`, `solari` and `proto.akoria.net`. A legacy vhost remains on :9443.
Document root	`/opt/solarinet/current/dashboard/public`
API	`/api/` → php-fpm on `unix:/run/php/solarinet85-fpm.sock`, with `SCRIPT_FILENAME` forced to `api/index.php`
php-fpm pool	`solarinet`, user `www-solari`. DB and service credentials are environment entries in `/etc/php/8.5/fpm/pool.d/solarinet.conf`.
Content-Security-Policy	`script-src 'self' 'unsafe-eval'`. `unsafe-eval` is needed by the in-browser JSX compiler.

Request lifecycle

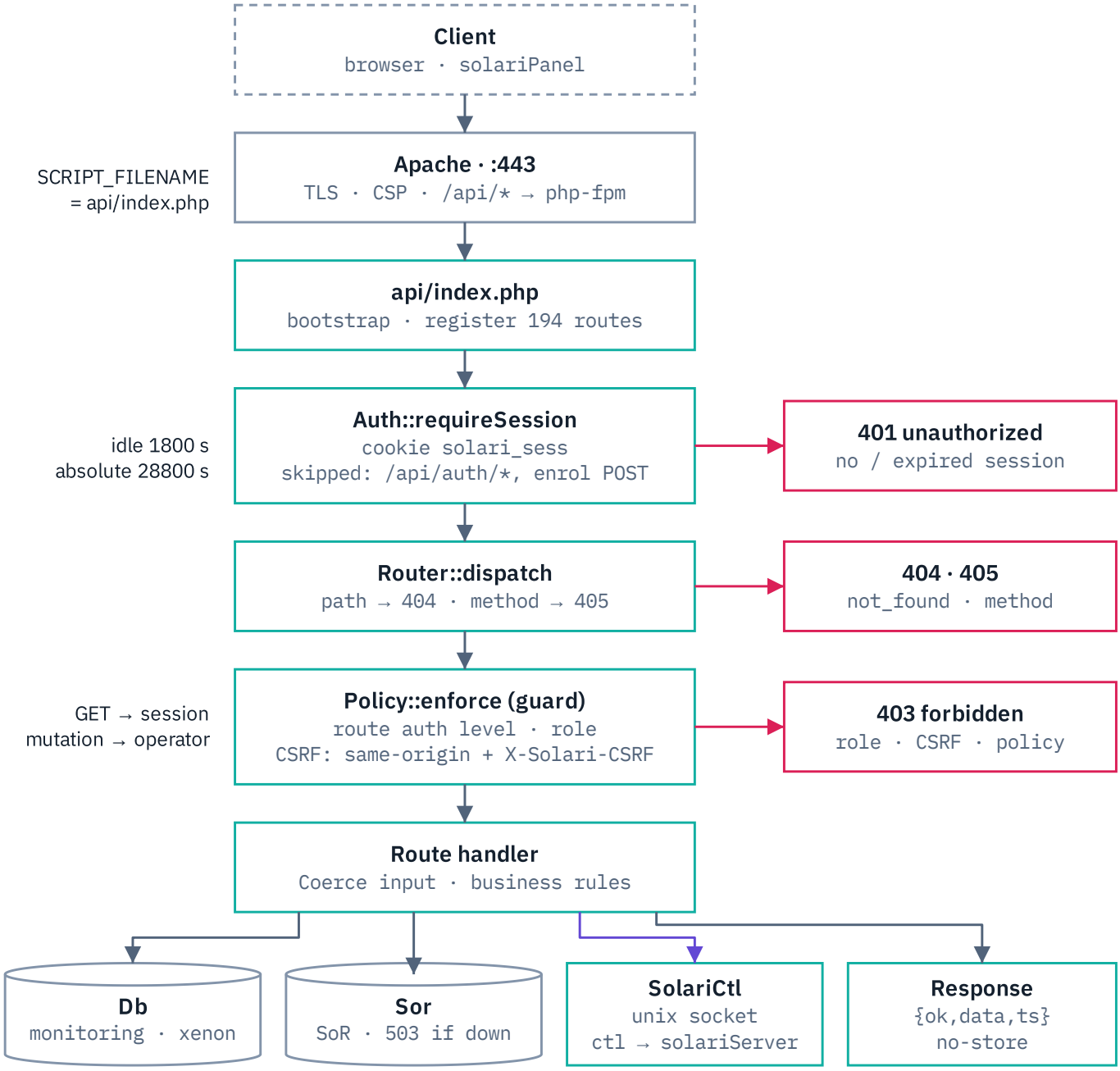


Figure 5 Lifecycle of a dashboard `/api` request: every route passes the session gate, the router and the policy guard before its handler runs.

`dashboard/api/index.php` runs the same steps for every request:

1. **Bootstrap** (`lib/bootstrap.php`) loads the libraries and configuration from the environment.
2. **Install the guard.** `$router->setGuard([Policy::class, 'enforce'])`.
3. **Register routes** from `routes/auth.php`, `routes.php` and `routes_mutations.php`, which in turn include the per-area files in `routes/`.

4. **Session gate.** `Auth::requireSession()` runs for every path except `/api/auth/*` and the two bearer-authenticated enrollment endpoints (`POST /api/enrollments` and `POST /api/enrollments/ack`). Failure is 401 `unauthorized`.
5. **Dispatch.** The router matches method and path (`{name}` segments). No path match is 404 `not_found`; a path match with the wrong method is 405 `method_not_allowed`.
6. **Policy.** The guard receives the matched route and the real request method, and runs before the handler: role check, then CSRF. Failure is 403 `forbidden`.
7. **Handler.** Validates input (`lib/Coerce.php`), reads or writes the monitoring DB or the SoR, or calls the ctl socket, and returns through `lib/Response.php`.

Every response is JSON with `Cache-Control: no-store`: `{"ok":true,"data":…,"ts":…}` or `{"ok":false,"error":{"code":…,"message":…}}`.

Sessions and sign-in

PROPERTY	VALUE
Cookie	`solari_sess`: HttpOnly, Secure, SameSite=Lax
Lifetimes	Idle 1,800 s; absolute 28,800 s; OIDC-backed sessions 3,600 s
Roles	`viewer` < `operator` < `admin`
OIDC	Authorization-code flow against Keycloak `https://sso.akoria.org:8443/realms/akoria`, client `solarinet-dashboard`. ID tokens verified against JWKS (RS256). AD groups map to roles (Domain Admins → admin). Roles are rechecked every 60 s.
Local accounts	bcrypt, break-glass only. `GET /api/auth/config` reported `oidcEnabled: true` and `localEnabled: true`. OBSERVED
Service principal	Local user `panel`, role viewer, used by solariPanel (Chapter 12)

Policy and CSRF

Each route declares one of four auth levels: `public`, `session`, `operator` or `admin`. A route that declares nothing gets `session` for GET and `operator` for any mutating method. Of the 194 routes in `docs/authz/AUTHZ_MANIFEST.json`, 6 are public, 86 need a session and 102 need an operator.

Every POST, PUT, PATCH and DELETE must pass two CSRF checks: the `Origin` (or, failing that, `Referer`) is same-origin, and the `X-Solari-CSRF` header equals the session's token, compared with `hash_equals`. The one exception is the `panel-service` bridge: the local `panel` viewer may post panel state when it sends no `Origin`, `Referer` or `Sec-Fetch-Site`, because it is a daemon and not a browser.

> **ADDING A ROUTE**
>
> Declare the auth level explicitly, add the route to the manifest and run the manifest test. Never mark a route `public` to get round a session problem. The six public routes are local sign-in, sign-in configuration, the OIDC start and callback, and the two enrollment endpoints, which carry their own bearer token. The API Expansion Manual walks through the full procedure.

Databases and the ctl socket

LIBRARY	TALKS TO	NOTES
`lib/Db.php`	Monitoring DB `solarinet` at 127.0.0.1:3306	PDO, exceptions on. Hardened deployments use a dedicated `solari_dash` grant.
`lib/Sor.php`	SoR `sor` at 10.1.0.200:3306	User `solari_dashboard`. Returns 503 if cesium is unreachable, so SoR pages fail while monitoring pages keep working. Writes stamp `source_id` with the `dashboard` source.
`lib/SolariCtl.php`	`/run/solari/solariCtl.sock`	One request per connection. Privileged actions go through `REQUEST_SUBMIT` and are then polled with `REQUEST_GET`.
`lib/Ca.php`, `lib/Forgejo.php`, `lib/Dig.php`	step-ca, Forgejo API, DNS	Read-only views. The Forgejo token is a pool environment entry.

The other libraries are `Auth`, `Oidc`, `Identity`, `Operator`, `Policy`, `Router`, `Response`, `Coerce`, `Label` (label images for inventory codes), `Rollup`, `ProvCatalog` and `DiscoveryScanInput`. Route files are grouped by area in `dashboard/api/routes/`.

The front end without a build step

The UI is React 18.3.1 with Babel standalone 7.29.0, both vendored in `dashboard/public/vendor/`. `index.html` declares 17 scripts of type `application/x-solarinet-jsx`. `boot.js` fetches each one, hashes it with SHA-256, and either finds the compiled output in CacheStorage (`solari-jsx-transform-v2`) or sends it to `transform-worker.js`, which runs Babel off the main thread. Compiled code runs by indirect `eval`, in order. The first load after a release compiles everything; later loads hit the cache.

Screens live in `screens.jsx` to `screens8.jsx`, plus `screens-panel.jsx`, `screens-rackwire.jsx` and `screens-prov.jsx`. The UI polls the API every 10 s. `/api/stream` returns 204: server-sent events were retired.

Service worker

`sw.js` caches as `solari-v15` OBSERVED: network-first for app code, cache-first for vendor files and fonts, and network-only for `/api`. Bump `CACHE_VERSION` whenever cached assets change shape, or installed iPads keep stale code. The worker has no `push` handler, so the push routes only store subscriptions; the notify `webpush` sender sends to them, but nothing in the page is written to show those messages. INFERRED

> **NO TOOLCHAIN**
>
> Do not add npm, a bundler or a transpile step. Edit the `.jsx` files directly; the browser compiles them. The design target is iPad Safari first, iPhone second.

CHAPTER 7

Data

Two databases

	MONITORING DB	SOR
Database	`solarinet` on xenon, 127.0.0.1:3306	`sor` on cesium, 10.1.0.200:3306; replica on benzene (`db.akoria.net`)
Engine	MariaDB 11.8.6 OBSERVED	MariaDB 12.3.2 OBSERVED
Holds	What the agents observe: nodes, host and probe state, history, alerts, ctl requests, discovery, topology	What the network should be: entities, addressing, DNS, functions, inventory, provisioning, RackWire plans
Size	About 40 tables; history tables partitioned; no triggers	60 base tables, 84 triggers OBSERVED
Migrations	`db/migrations` 001–037 (latest `037_node_certcn_role_prefix.sql`)	`netdb/sor/migrations` (001, 014, 015, 020–037)
Written by	solariServer, solariSnmpPoll, dashboard, alertbridge (checkpoint)	Dashboard, sync daemons, unifipolld, provisioning helpers, RackWire

The two share no schema, no foreign keys and no migration numbering. Monitoring migration 037 and SoR migration 037 are unrelated. Retention for the monitoring history is in `db/retention.sql` and `db/roll_history.sql`.

Migrations and the ledger

`tools/migrate.py --database sor|monitoring` applies migrations and records each in `solari_migration_ledger` (`migration_id`, SHA-256 `checksum`, `state` applying/applied, `started_at`, `applied_at`). `--check` reports pending work; `--verify-live` compares the live schema with the expected one. A migration whose checksum has changed after it was applied is an error: write a new migration instead.

The SoR ledger records 024 to 037 as applied; the last was `037_inventory_items_closeout` at 2026-11-03 16:18:43 (server local time). OBSERVED Migrations before 024 predate the ledger. The monitoring DB's ledger was not inspected. UNVERIFIED

Rollback scripts for SoR migrations are in `netdb/sor/rollback/`. `tools/export_schema.py` writes schema snapshots and `tools/audit_schema.py` checks them. CI rehearses SoR migrations 033 to 037 against a fresh MariaDB on every run.

Provenance

Every SoR row that someone or something asserts carries provenance columns: `source_id` (→ `sources`), `asserted_kind` (`human` or `machine`), `asserted_at`, `created_at`, `updated_at` and, for soft deletion, `deleted_at`. The live sources are `manual`, `dashboard`, `netdb`, `samba_ad`, `bind`, `forgejo`, `solarinet_monitor`, `unifi`, `pihole`, `rackwire`, `provisioner` and `scanner-ingest`. OBSERVED

The rule for conflicting writes: a human assertion wins for intent fields (names, roles, criticality, lifecycle); a machine assertion wins for observed fields (MAC, last seen, firmware). Writers stamp their own source and must not overwrite a human-asserted intent field from a machine path.

Soft-deleted rows stay in the table. Uniqueness among live rows uses a generated `live_flag` column (1 when `deleted_at` is null, otherwise null) inside the unique key, so a deleted row never blocks a new one with the same natural key.

Types and conventions

- **IP addresses are native `INET6`.** IPv4 is stored v4-mapped. Read with `CAST(col AS CHAR)`; write with `CAST(? AS INET6)`. `INET6_NTOA` on the native type and a bare `INET6_ATON` on IPv4 text both fail silently; `INET6_ATON('::ffff:a.b.c.d')` with an explicit v4-mapped prefix works, and is what the discovery reconciler uses.
- **Criticality** is `functions.criticality`, TINYINT 1–5 with 1 the most critical, default 3, enforced by a CHECK constraint.
- **Lifecycle** lives on `entities.lifecycle`. Retired rows are tombstoned, not deleted.
- **Edit conflicts** are optimistic: a client sends back the `updated_at` it read; the handler locks the row (`SELECT … FOR UPDATE`) and returns 409 `conflict` if it changed.

Triggers

The SoR uses triggers for three jobs. CDC triggers feed the sync pipeline (Chapter 8). Integrity triggers enforce rules that a constraint cannot express. Bridge triggers keep old and new inventory tables consistent while legacy writers still exist.

MIGRATION	TRIGGERS	PURPOSE
023, 024, 026, 030	`trg_<table>_ai/_au/_ad` on 23 tables	Write a change row to `sor_outbox`
014	`trg_alloc_no_double_commit_ins/upd`, `trg_alloc_tray_move`	A unit can be committed to only one project; committing moves it to the project tray and sets `stock_status`
028	entity ↔ hardware-unit link	Keep `entities.hardware_unit_id` and `entity_hardware_unit_links` in step
029	`prov_runs` guards	Provisioning runs are append-only: finished rows are immutable
033, 035	`trg_hw_models_catalog_ins/upd`, `trg_hw_units_installed_target_ins/upd`, `trg_product_types_no_software_install_target`	Give every hardware model a catalog type; validate “installed on” targets
036	location level and cycle triggers	Roots must be `property`; the levels must nest property → floor → room → section; storage units may sit anywhere; no cycles
037	`trg_hw_units_installed_target_ins/_upd` (redefined)	Require `product_type_id` on every unit and keep it consistent with the now-nullable `model_id`: a writer that sets only `model_id` still derives the catalog type, and a mismatch is refused

> **TRIGGERS AND BULK LOADS**
>
> Every write to a CDC table produces an outbox row and, later, a DNS or AD apply. A bulk update of `ip_addresses` can trigger thousands of applies. Batch large changes, and check the outbox depth before and after.

Domain models

The SoR falls into four domains. The inventory domain is drawn in Chapter 11; RackWire's tables are described in Chapter 12.

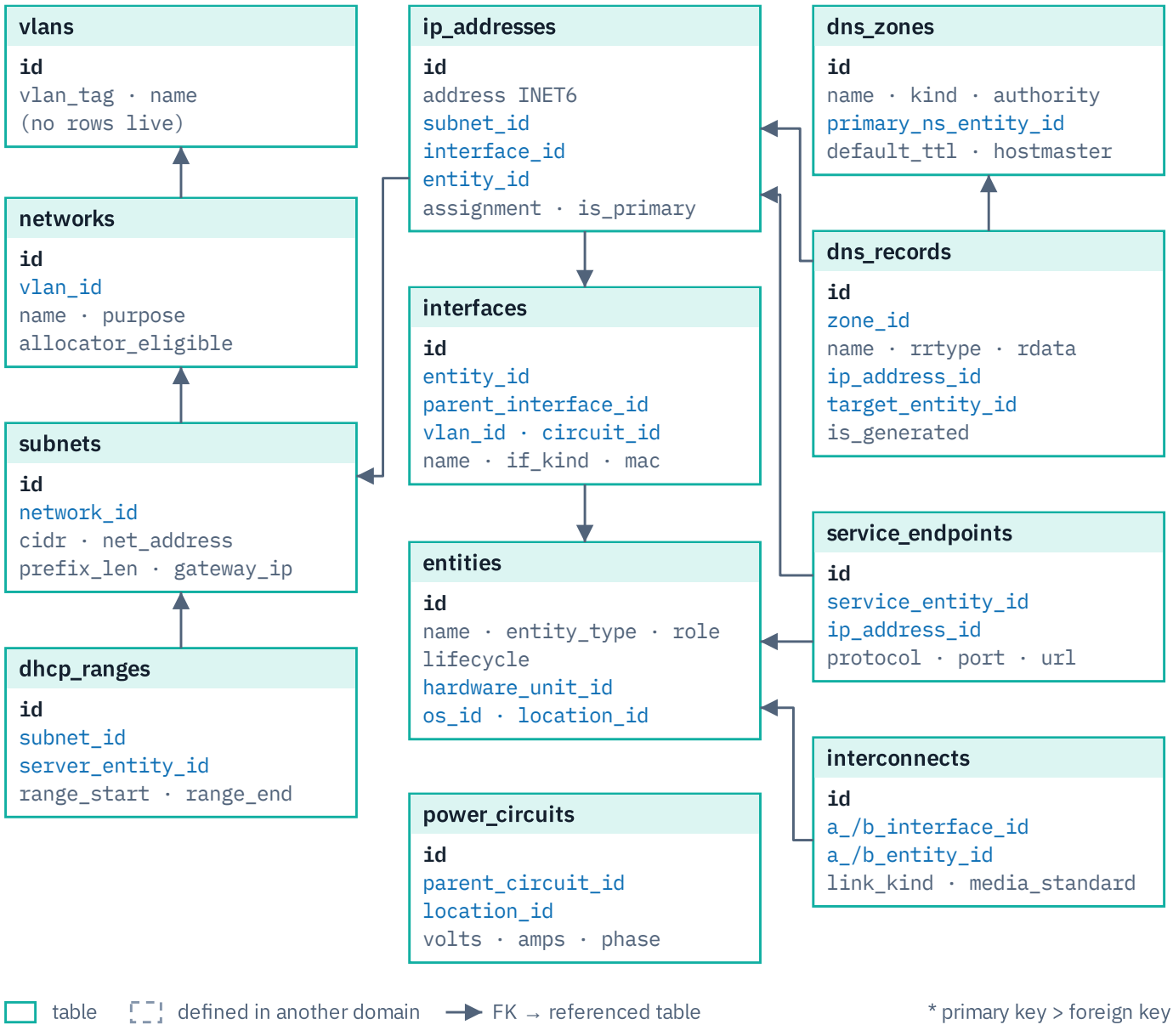


Figure 6 Network domain: addressing, interfaces, DNS, endpoints, cabling and power.

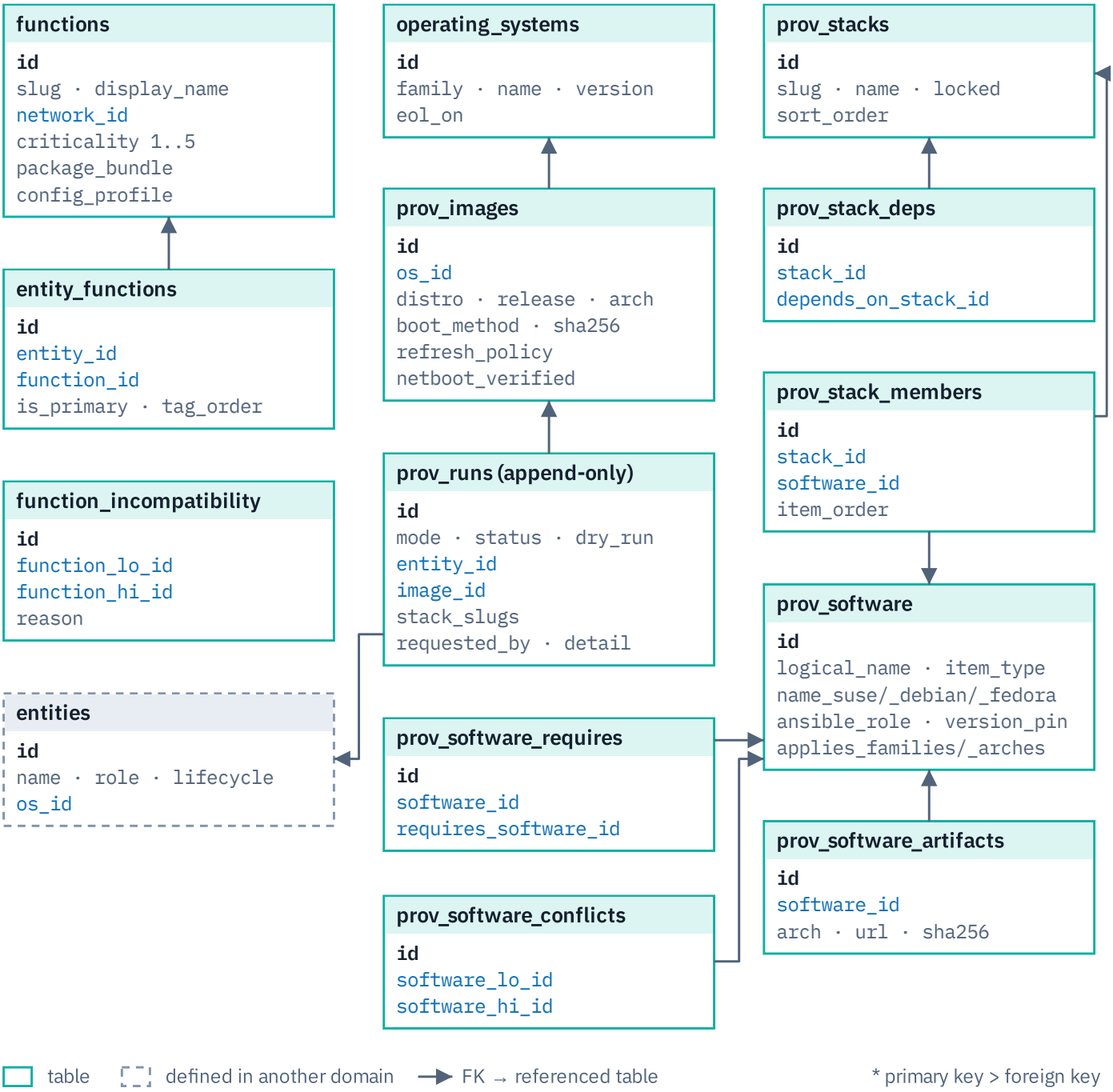


Figure 7 Functions and provisioning domain: function assignment, images, software catalog, stacks and runs.

CHAPTER 8

SoR sync and messaging

Why change data capture

The SoR is the single place where intent is written. Systems that act on that intent (BIND, Samba AD) must follow it without the dashboard or any other writer having to know about them. SolariNet does this with a transactional outbox: the same transaction that changes a row writes a change record, and separate daemons turn those records into actions.

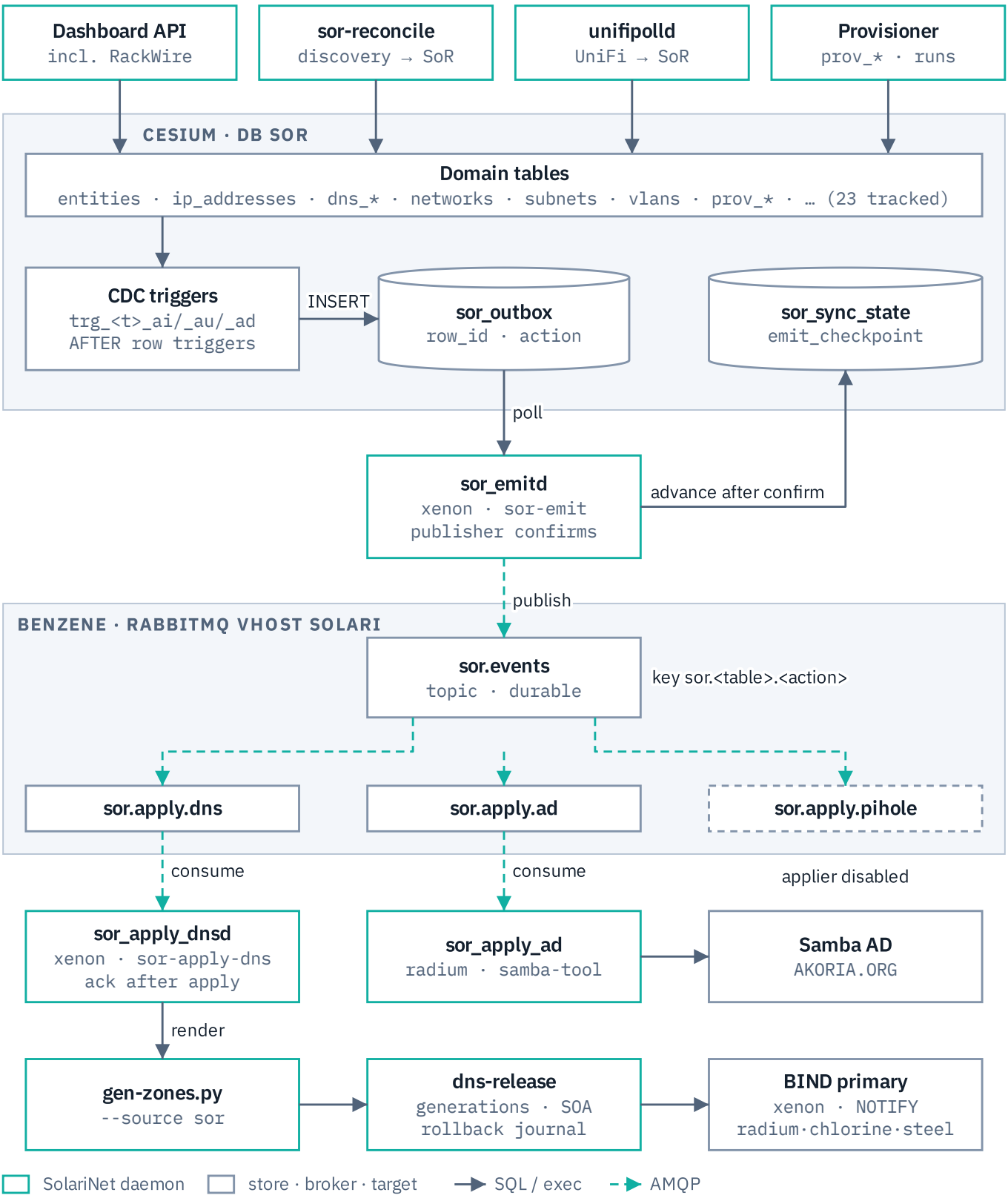


Figure 8 SoR change-data-capture: triggers to outbox, outbox to RabbitMQ, queues to appliers, appliers to DNS and Active Directory.

Outbox and emitter

CDC triggers on 23 tables (`dhcp_ranges`, `dns_records`, `dns_zones`, `entities`, `entity_functions`, `external_refs`, `functions`, `function_incompatibility`, `interfaces`, `ip_addresses`, `networks`, `sources`, `subnets`, `vlans` and the nine `prov_*` tables) insert `(table_name, row_id, action)` into `sor_outbox`. OBSERVED

`deploy/sorsync/sor_emitd.py` (unit `sor-emit` on xenon) polls the outbox and publishes each row to the durable topic exchange `sor.events` with routing key `sor.<table>.<action>` and a JSON body `{outbox_id, table, row_id, action, ts}`. Messages are published `mandatory` (except for tables listed as optional) with publisher confirms, and the checkpoint `emit_checkpoint` in `sor_sync_state` moves forward only after the broker confirms. If the emitter dies between publishing and checkpointing, the same rows are sent again: delivery is at-least-once, and consumers must be idempotent.

Appliers

DAEMON	QUEUE	WHERE	ACTION
`sor_apply_dnsd.py`	`sor.apply.dns`	xenon, unit `sor-apply-dns`	For changes to entities, ip_addresses, interfaces, dns_records, dns_zones, networks or subnets: render all zones with `netdb/gen-zones.py --source sor` and hand the render to the dns-release engine
`sor_apply_ad.py`	`sor.apply.ad`	radium, as root UNVERIFIED	Apply DNS and host changes to Samba AD with `samba-tool`
`sor_apply_pihole.py`	`sor.apply.pihole`	disabled	Retired with the Pi-holes

All appliers share `delivery.py`: a message is acknowledged only after the apply succeeds (`consume_after_apply`), every apply is an idempotent full re-render rather than a delta, and each applier reconciles at startup and hourly, so a lost message is repaired within the hour. Database grants for each daemon are in `deploy/sorsync/sor-runtime-grants.sql`.

dns-release

`deploy/dns-release/release.py` owns the managed BIND zones on xenon. Each render becomes a generation directory under `/etc/bind/solarinet-releases/`, named by a content hash. The engine validates every zone and the whole BIND configuration, records the serial high-water marks in a journal, switches the `/etc/bind/solarinet-current` symlink, reloads `named`, and then requires authoritative SOAs plus an expected A or PTR record for each zone. If verification fails it rolls back by writing a new generation whose serials move past the failed attempt, so AXFR secondaries follow the rollback too. `named.conf.local` includes `/etc/bind/solarinet-current/named.conf.managed`, and the current link was last switched on 2026-11-03. OBSERVED

The engine's configuration is `/etc/solarinet/dns-release.json` (root-owned; the CLI refuses writable configuration). Candidates must be private directories under its `incomingRoot`, owned by the configured candidate user. Zones that are not managed stay outside the release.

Inbound writers

Two daemons write observed state into the SoR rather than reading intent out of it:

- **`sor_reconcile_discovery.py`** (unit `sor-reconcile`) polls the monitoring DB's discovery and node tables and creates or updates SoR entities and addresses, stamped `scanner-ingest` or `solarinet_monitor`. It writes to CDC tables, so its changes flow on to DNS.
- **`unifipolld`** (`deploy/unifi`) polls the UniFi controller on chemistry every 15 s and writes clients, interfaces and addresses with source `unifi`, using the `[sor_unifi]` configuration section.

Both are machine writers, so they must respect human-asserted intent fields (Chapter 7).

CHAPTER 9

Alerting and notification

From probe to phone

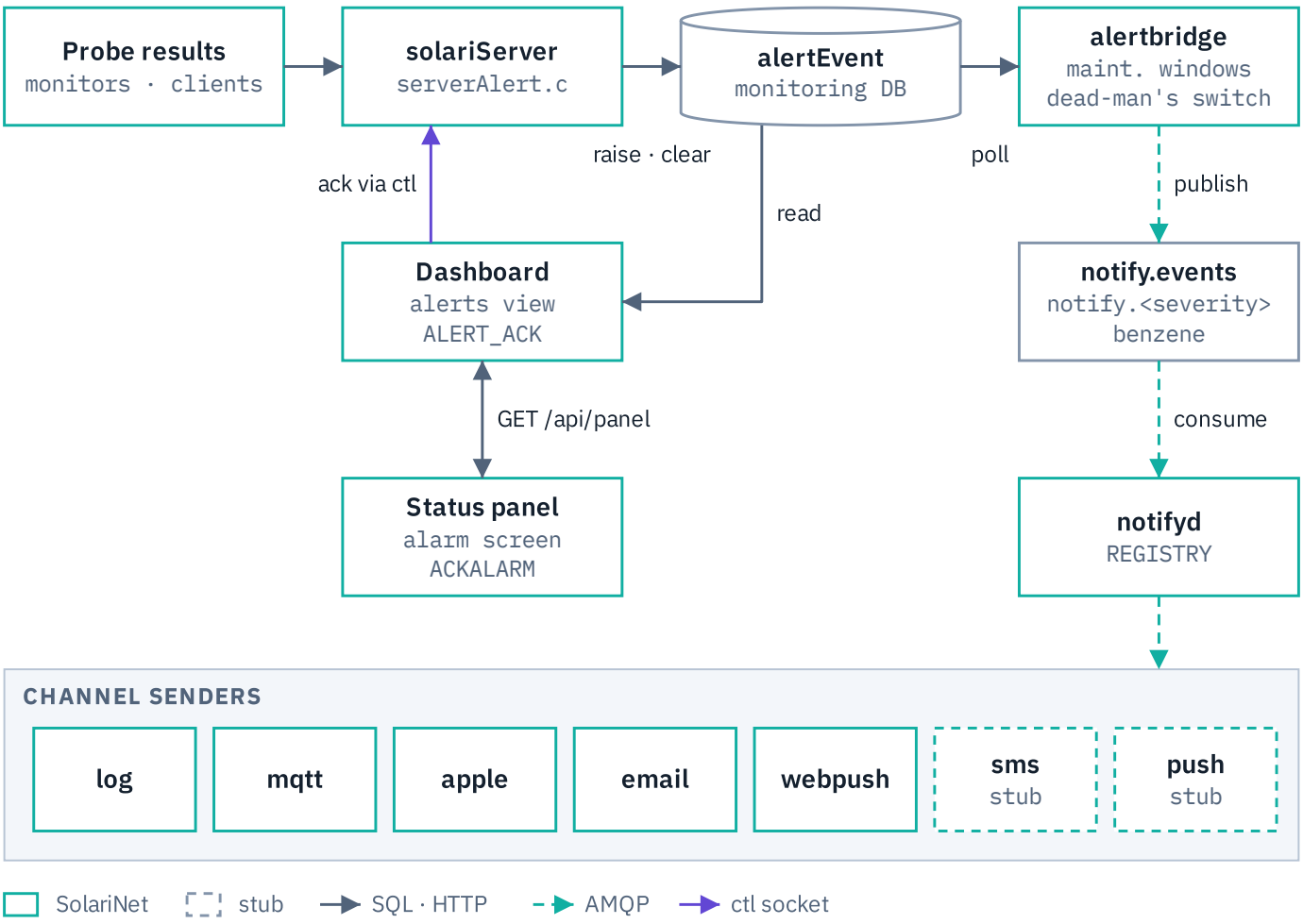


Figure 9 Alert and notification path.

1. **Raise.** `serverAlert.c` in solariServer evaluates alert rules against incoming reports. It writes an `alertEvent` row with a disposition (`publish` or `suppress`, using the 5-tier criticality and any acknowledgement) and also PUBs the event.
2. **Bridge.** `deploy/alertbridge/alertbridge.py` polls `alertEvent` from a checkpoint, drops events for hosts inside a `maintenanceWindow`, and publishes the rest to `notify.events` with routing key `notify.<severity>`, waiting for confirms. Already-checkpointed events are never published again.
3. **Fan out.** `deploy/notify/notifyd.py` consumes `notify.events` and routes each message, by severity, to the senders enabled in `notify.conf`.
4. **Acknowledge.** The dashboard sends `ALERT_ACK` through the ctl socket. The status panel can acknowledge the active alarm with its ACKALARM control (Chapter 12).

The dead-man's switch

alertbridge also watches the watchers. Each cycle it computes how long ago each node last reported, from `hostCurrent` (clients) and `probeCurrent` (monitors). A node that was reporting and has been silent for more than max(120 s, 3 × its sample interval) produces one synthetic `crit` “stopped reporting” message; when it reports again an `info` recovery follows. Nodes in a maintenance window are skipped. This is the check that would have caught cesium's 26-hour silence in July.

Senders

NAME	MODULE	TRANSPORT
log	`senders/log.py`	Writes to the journal
mqtt	`senders/mqtt.py`	Publishes to Mosquitto on benzene
apple	`senders/apple.py`	SSH to an always-on Mac and send an iMessage with `osascript`; iCloud fans it out to the operator's devices
email	`senders/mail.py`	SMTP. The module is `mail.py` so that it does not shadow the standard library's `email`.
webpush	`senders/webpush.py`	Web Push (VAPID) to every stored browser subscription
sms	`senders/sms_tachyon.py`	Stub
push	`senders/push.py`	Interface-only stub, kept so configuration that names it does not crash

The registry is `deploy/notify/senders/__init__.py`. `deploy/notify/README.md` lists an older set of senders; trust the registry.

> **ALARM TESTS ARE LOUD**
>
> A synthetic critical event goes to every enabled channel, including the panel, which flashes and sounds. Warn the household before testing, and prefer the `log` sender or a maintenance window when developing.

CHAPTER 10

Provisioning

From bare metal to a running stack

Provisioning takes a host with an empty disk to an enrolled SolariNet node with its software stack applied. The SoR describes what each host should be (`entities`, `entity_functions`, `prov_images`, `prov_stacks`); `deploy/fleet` turns that into boot files, installer answers and Ansible runs.

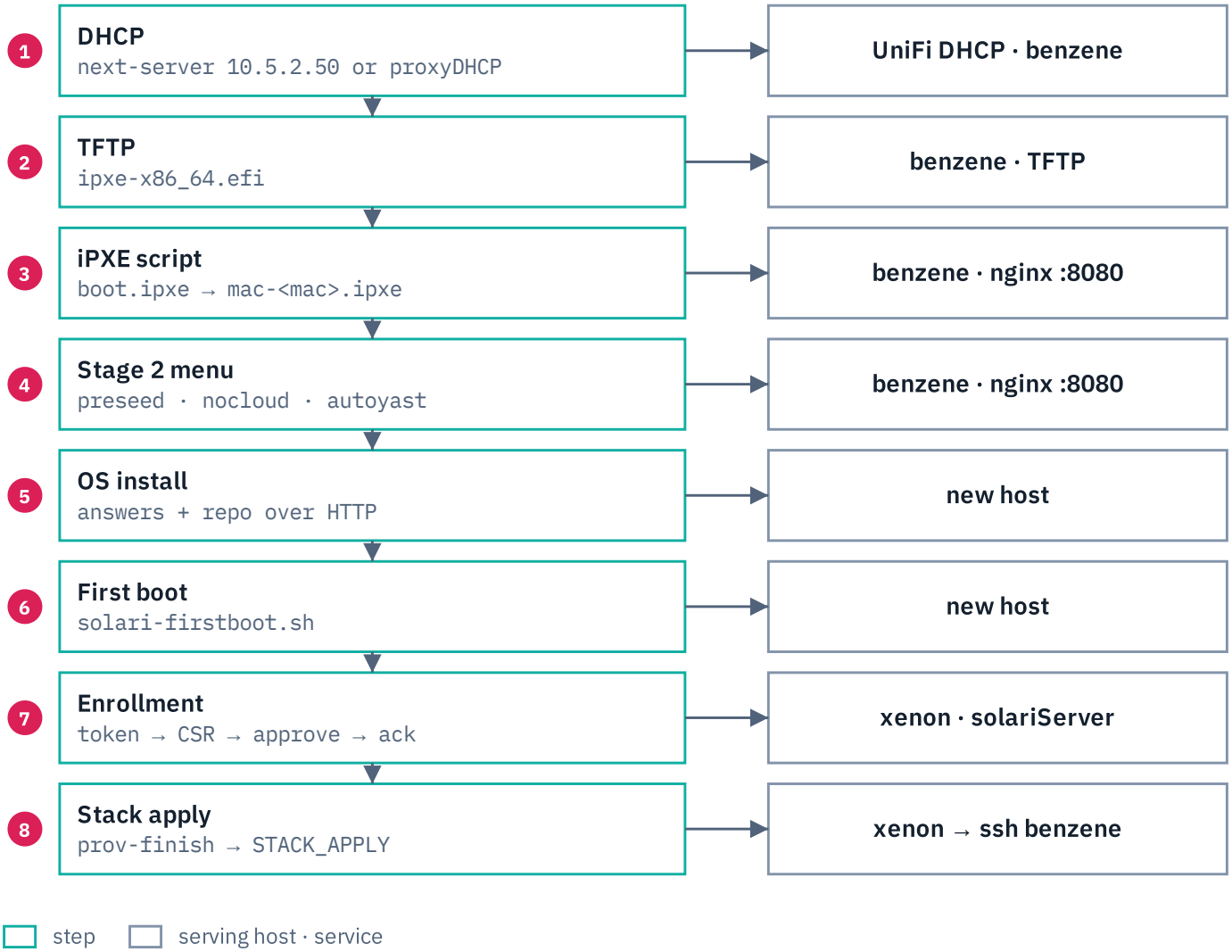


Figure 10 Provisioning sequence from network boot to applied stack.

Network boot

The netboot tree is rendered by the fleet helpers from the SoR image and host rows (`deploy/fleet/netbootlib.py` owns every path and every iPXE line) and rsynced to benzene, which serves it over HTTP on nginx `http://10.5.2.50:8080/` and serves the iPXE loaders over TFTP. DHCP points machines at it

in one of two ways: a proxyDHCP on benzene, or UniFi DHCP options with next-server 10.5.2.50 and filename `ipxe-x86_64.efi`. Which one is live was not checked. **UNVERIFIED**

The loader runs an embedded script that chains `boot.ipxe`, then a per-machine `mac-<mac>.ipxe` (stage 1), then a stage 2 menu offering the installer for the image's family (preseed for Debian, Ubuntu autoinstall through a NoCloud data source, AutoYaST for openSUSE, which installs from the upstream repository). Stage 3 is the local disk. Images are fetched and checksummed by `fetch-images.sh` and `image-refresh.py`; `prov_images.os_id` links each image to its distribution row in `operating_systems`, so reimaging a host also sets its OS in the SoR.

First boot and enrollment

The installer drops `solari-firstboot.sh`, which installs solariClient and writes `/etc/solari/PENDING_ENROLLMENT`. Enrollment then follows the token flow in Chapter 5. `prov-finish.py`, run by `solari-prov-finish.timer` on xenon, notices hosts that have finished installing and enrolled, and queues the stack apply.

STACK_APPLY and Ansible

`STACK_APPLY` is a privileged ctl verb, so it arrives through the ctl queue. solariServer starts `stack-apply.py`, which:

1. generates the host's stack variables from the SoR with `stackgen.py --from-db`;
2. inserts a `prov_runs` row (append-only, see Chapter 7);
3. runs `ansible-playbook playbooks/site.yml --tags stack` on benzene through `provlib.remote_run`, which uses `ssh` in BatchMode, first with `--check --diff` and then for real;
4. records the result on the run row.

The Ansible tree lives in `deploy/fleet/ansible` and is published to `/srv/solari-provision/ansible` on benzene by `deploy/fleet/sync-ansible.sh` (exit 0 in sync, 1 drift, 2 check failed). On benzene the published tree is a set of symlinks into a release. Roles: `base-os`, `common-stack`, `dev-tools`, `directory-join`, `dns-secondary`, `shared-home`, `solarinet-client` and `stack-items`. steel became a BIND secondary through the `dns-secondary` role in this way.

The other provisioning verbs follow the same pattern: `FLEET_PROVISION` → `fleet-provision.sh`, `FLEET_IMAGE` → `fleet-image.sh`, `IMAGE_REFRESH` → `image-refresh.py`, `PROV_STAGE` → `prov-stage.py`.

> **UNSIGNED PROVISIONING IS FAIL-CLOSED**
>
> `deploy/fleet/provisioning-guard.sh` blocks provisioning helpers until the signed provisioning chain (finding F-07) exists. Setting `SOLARI_ALLOW_UNVERIFIED_PROVISIONING=1` overrides it and is meant only for an isolated lab network. `prov-store-guard.sh` checks the image store on the provisioning host before helpers read or write netboot payloads.

CHAPTER 11

Inventory I1

Scope of Inventory I1

Inventory began in July 2026 as barcode tagging for network assets. Its remit was widened on 2026-11-01 (`docs/Inventory_Remit_2026-11-01.md`) to cover anything physical or licensed: one product catalog across hardware, peripherals, components, software, accessories and consumables; flexible locations with hundreds of reusable project containers; image-only labels; and license references, never license keys. I1, the first phase, shipped as SoR migrations 033 to 036, with the close-out in 037, and is live. OBSERVED

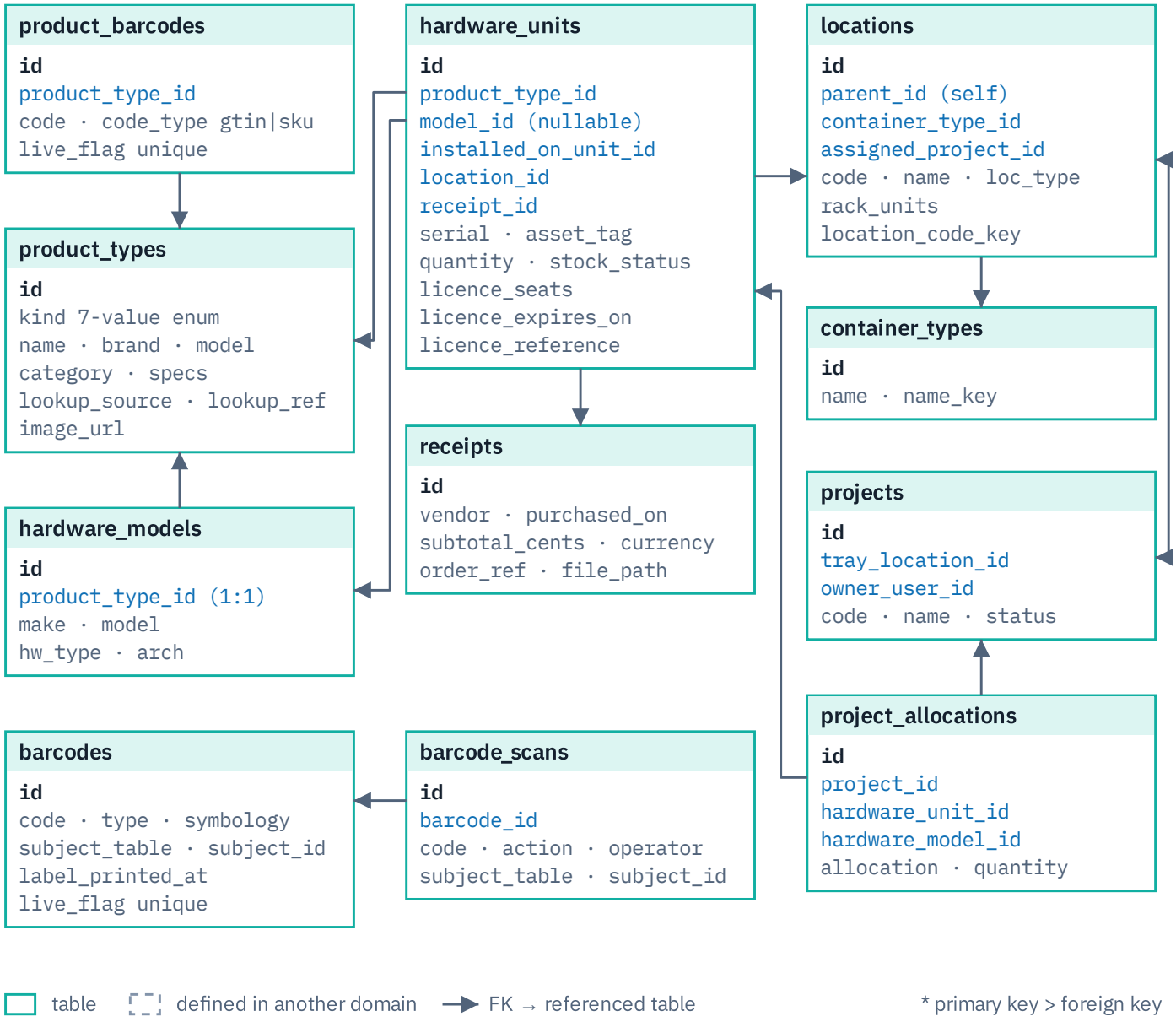


Figure 11 Inventory domain (Inventory I1). Arrows run from the referencing table to the referenced one.

Data model

TABLE	ROLE
`product_types`	The catalog. `kind` is one of hardware, peripheral, component, software, accessory, consumable or other. Optional `lookup_source` and `lookup_ref` record where details came from; only free lookups are used.
`hardware_models`	Legacy model rows, now one-to-one with a catalog type; a bridge trigger creates the type for legacy writers.
`hardware_units`	Individual items or stock lines: `quantity`, `serial`, `asset_tag`, `location_id`, `stock_status` (in_stock, allocated, deployed, consumed, retired), `receipt_id`, `unit_cost_cents`.
`locations`	A tree: `property` → `floor` → `room` → `section`, with `storage_unit` allowed at any level. Storage units have a `container_type` and may be assigned to a project.
`projects`, `project_allocations`	What a unit or model is reserved for. Committing a unit moves it into the project's tray location.
`barcodes`, `barcode_scans`	System-issued codes attached to any subject (polymorphic: `subject_table` + `subject_id`, no foreign key), and an audit log of every scan.
`product_barcodes`	Manufacturer GTINs and SKUs that identify a catalog type.
`receipts`, `inventory_snapshots`	Purchase records (file by path) and point-in-time exports.

Items, software and licenses

`hardware_units.model_id` is nullable: an item belongs to a catalog type through `product_type_id` and needs a hardware model only when it is hardware. Items are created from a catalog type with **Add item(s)**, which takes a quantity (up to 50 separate items, or a count on one consumable item), a location and, when adding a single item, an optional serial number and asset tag.

Software items carry `licence_seats`, `licence_expires_on` and `licence_reference`. The reference is a pointer to where the license is kept (a vault entry or a document path), never the key itself. A software item can be marked as installed on another unit through `installed_on_unit_id`; the target must not itself be software, and triggers enforce that.

The Components picker hides retired catalog types. Location codes are unique across all live locations, not only among siblings.

Edit-conflict timestamps carry microseconds, so two saves within the same second are still told apart. A database deadlock during an inventory write returns 409 with a “try again” message instead of a 500.

Routes

Inventory routes live in four files under `dashboard/api/routes/`: `inventory.php`, `product_catalog.php`, `location_admin.php` and `inv_codes.php`. Reads need a session; writes need an operator.

AREA	ROUTES UNDER /API/INVENTORY/
Catalog	product-types (list, create, read, update, delete); models; product-barcodes
Items	components (list, create, read, update, delete) and components/{id}/entity-link
Locations	locations, {id}/contents, {id}/merge-preview, {id}/merge, {id}/move, {id}/project, bulk-storage-units; container-types
Projects	projects, {id}/rollup, {id}/allocations
Codes	codes, codes/history, codes/{id}, codes/{id}/label, codes/{id}/printed, codes/lookup, codes/associate, codes/move, codes/receive
Receiving	receipts, receipts/{id}, receipts/{id}/scan
Views	/api/inventory, allocations, shopping

Every write runs in a transaction and locks the rows it changes. The catalog, location and container-type edit and move handlers also check the client's `updated_at`; component edits and deletes do not yet. Writes stamp the `dashboard` source with `asserted_kind='human'`, and maps constraint violations to 409. Labels are rendered server-side as images by `lib/Label.php`.

CHAPTER 12

RackWire and the status panel

RackWire

RackWire is a rack and cabling planner developed in its own repository and included as the submodule `dashboard/public/rackwire` (`../RackWire.git`). It is a single-file app; the dashboard embeds it in a same-origin iframe (`screens-rackwire.jsx`) and talks to it only through a documented `postMessage` bridge. The parent page checks both the message origin and the sending frame, because same origin alone would also admit other frames on the dashboard's origin.

Twelve routes under `/api/rackwire/` back it:

ROUTE	PURPOSE
`GET plans`, `plans/{id}`, `plans/{id}/versions`	Saved plans and their version history (`rw_plans`, `rw_plan_versions`)
`POST plans`, `plans/{id}/versions`, `plans/{id}/restore`, `plans/{id}/delete`	Create, save a version, restore an old version, soft-delete
`GET library`, `POST library/{modelId}/groups`	Equipment library from the SoR hardware models, with port groups in `rw_model_groups`
`GET telemetry`	Live port state from the monitoring DB (SNMP interface tables)
`GET sor/interconnects`, `POST commit`	Read the SoR cabling, and commit a plan's cables to `interconnects`

Writes are stamped with the `rackwire` source. The submodule pin is part of the release: `package-release.py` refuses to package if the checked-out submodule does not match the recorded commit. The standalone RackWire build intended for a third party is a separate product and not part of SolariNet.

> **SUBMODULE**
>
> Do not commit changes inside `dashboard/public/rackwire` from the SolariNet repository. Change RackWire in its own repository, then bump the pin here in a separate commit.

Status panel

The status panel is a Pimoroni Galactic Unicorn (RP2350, 53 × 11 LEDs) on lithium. It shows twelve screens in four themes, ticks every 40 ms, and turns pink and loud when a critical alarm is active. Three pieces make it work:

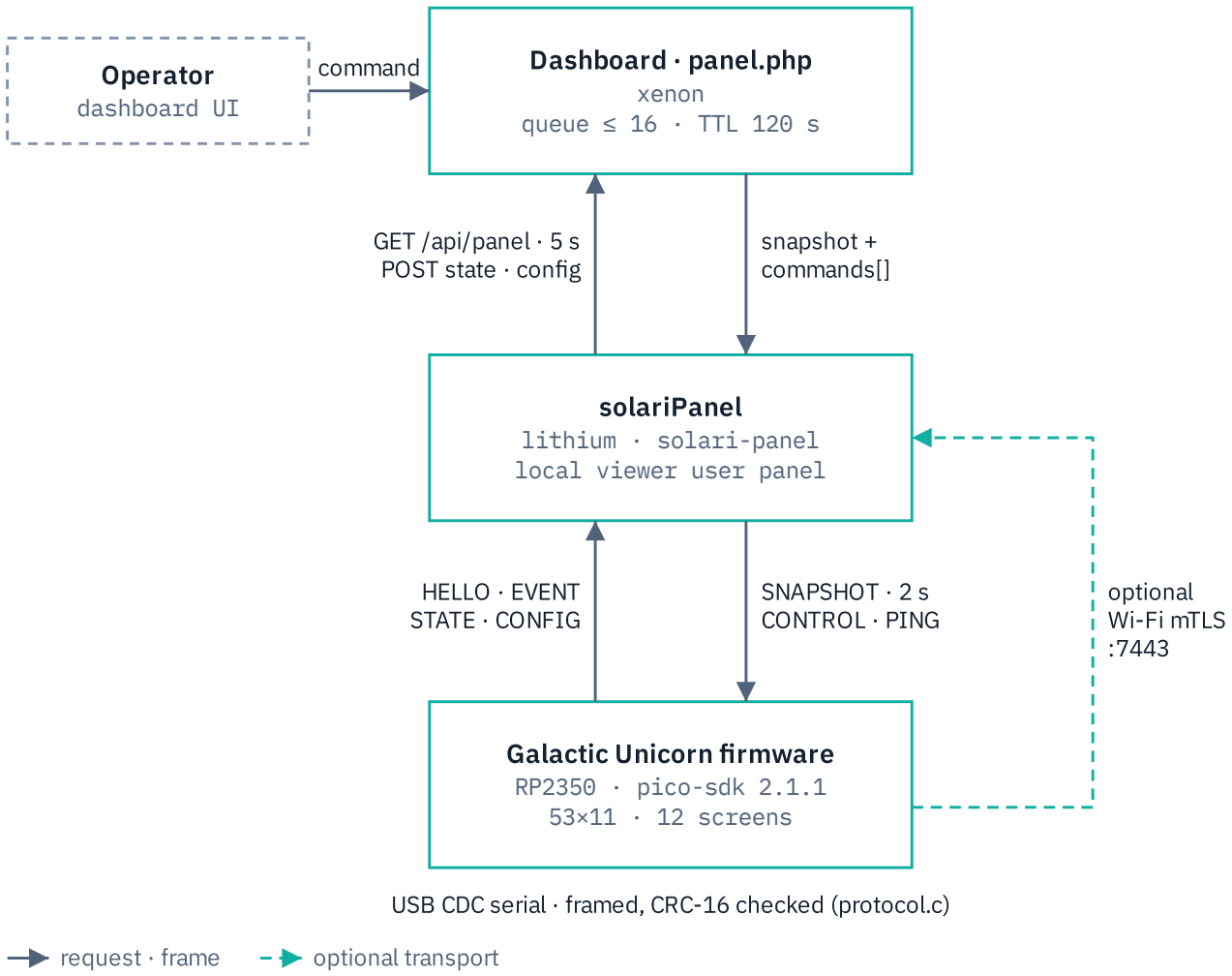


Figure 12 Status panel chain from operator command to LED matrix.

PIECE	SOURCE	NOTES
Panel API	`dashboard/api/routes/panel.php`	`GET /api/panel` returns a compact snapshot and, for the service principal, pending commands in `data.commands[]`. `POST /api/panel/state` and `/config` receive the panel's state; posting state marks commands up to `lastCmdId` as applied. `POST /api/panel/command` is operator-only; at most 16 commands may be pending (409 `queue_full`) and each expires after 120 s.
solariPanel	`status-panel/daemon`	C, own Makefile, libcurl and OpenSSL. Unit `solari-panel.service`, `User=jason`, group `dialout`. Signs in as the local `panel` viewer, polls every 5 s and writes a SNAPSHOT frame to the panel every 2 s. It forwards a queued command as a CONTROL frame only after the panel's STATE has reported that it supports CONTROL.
Firmware	`status-panel/firmware`	C on pico-sdk 2.1.1. Shares `status-panel/protocol.c` with the daemon.

Serial protocol

```
0xA5 0x53 | version 0x01 | type | u16 length (LE, ≤ 2048) | payload | CRC-16/CCITT
```

HOST → PANEL	PANEL → HOST
SNAPSHOT 0x01, PING 0x02, HELLOREQ 0x03, CONTROL 0x04, PROVISION 0x05	HELLO 0x81, EVENT 0x82, LOG 0x83, STATE 0x84, CONFIG 0x85, PROVACK 0x86

CONTROL kind 5 is ACKALARM. Alarm episode identifiers are split into ranges so the three producers never collide: below `0x80000000` are server event IDs, `0x80000000`–`0xBFFFFFFF` are server CRC episodes, and `0xC0000000` and above are firmware-local.

A standalone Wi-Fi mode, merged in September, lets the panel connect without USB: solariPanel listens on :7443 for raw mutual-TLS connections whose client certificate is issued by step-ca and whose SAN is on an allowlist (`panel-01.panel.akoria.net`). The firmware for it has not been flashed and no certificate has been issued, so the live panel runs over USB only. **UNVERIFIED**

`status-panel/README.md` calls the daemon `solari-paneld`; the binary and unit are `solariPanel` and `solari-panel.service`.

CHAPTER 13

Security boundaries

Trust zones

SolariNet runs on a private network behind a firewall, for one household. Security work is weighed against that: strong boundaries where a mistake would be expensive (privileged actions, identity, certificate issuance), and plain pragmatism elsewhere. The figure shows the boundaries that code enforces; Table 13.1 gives the authentication at each numbered crossing.

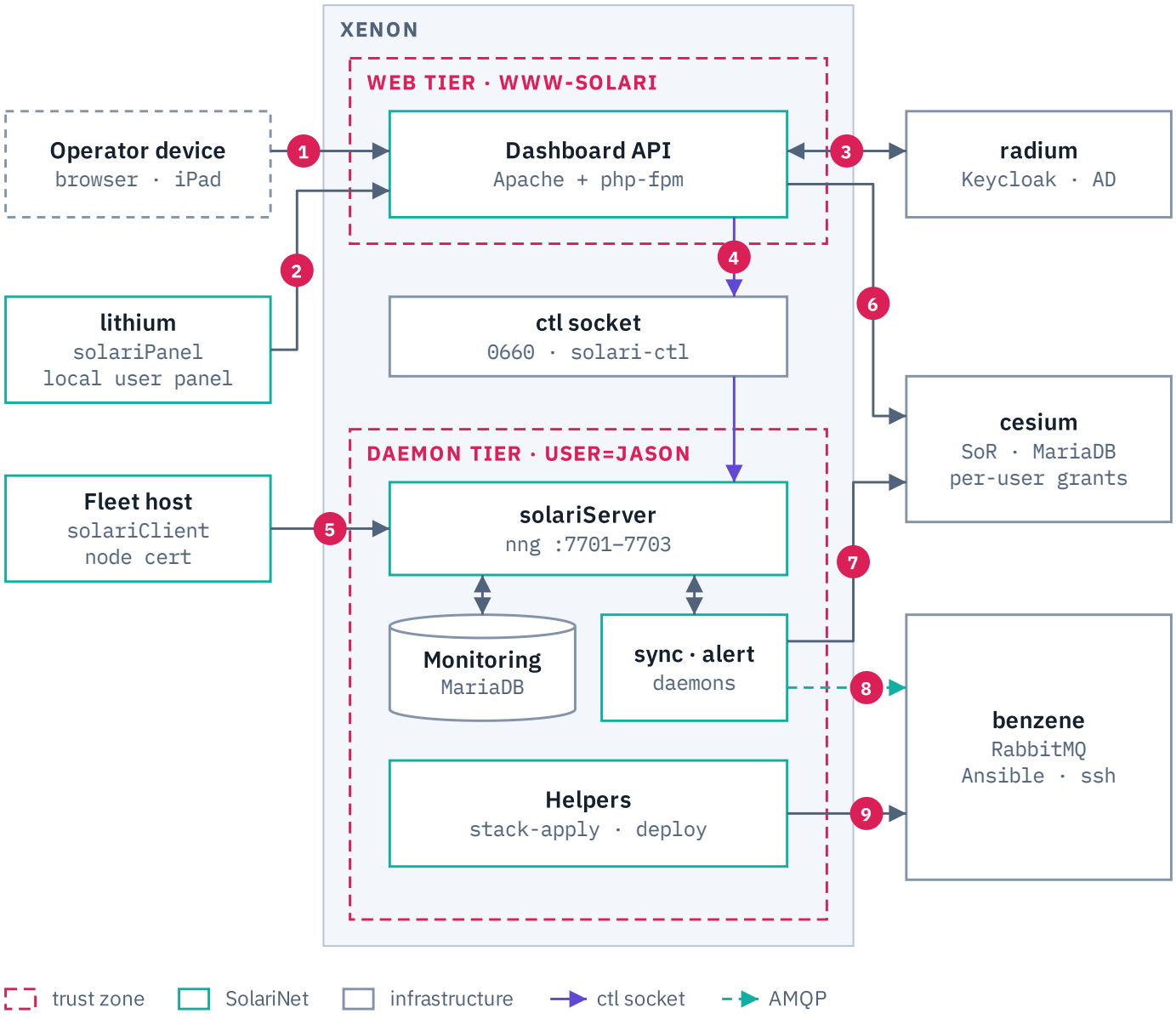


Figure 13 Trust zones and numbered crossings; Table 13.1 gives the authentication used at each crossing.

Table 13.1 · Authentication at each numbered crossing

#	CROSSING	TRANSPORT	AUTHENTICATION
1	Operator browser → Dashboard	HTTPS :443	OIDC (Keycloak) session cookie `solari_sess`; role per route; CSRF token + same-origin on writes
2	solariPanel (lithium) → Dashboard	HTTPS :443	Local `panel` viewer session; CSRF bridge for a client with no browser headers
3	Dashboard ↔ Keycloak	HTTPS :8443	OIDC authorization-code flow with a client secret (pool environment); ID token checked against JWKS
4	Dashboard → ctl socket → solariServer	AF_UNIX	File mode 0660, group `solari-ctl`; `SO_PEERCRED` class; privileged verbs only through the queue, re-bound to the requesting operator
5	Fleet agent → solariServer	nng over TLS :7701–7703	Mutual TLS with SolariNet CA certificates; CN must match node ID and approved record
6	Dashboard → SoR	MariaDB :3306	Per-user grant `solari_dashboard`; TLS not verified UNVERIFIED
7	Sync and alert daemons → SoR	MariaDB :3306	Per-daemon grants from `sor-runtime-grants.sql`
8	Daemons → RabbitMQ	AMQP :5672	Username and password, vhost `solari`; TLS not verified UNVERIFIED
9	Provisioning helpers → benzene	SSH	Key-based, BatchMode, no password prompts

Rules that hold the boundaries

- **PHP never holds operator authority.** It may only ask; solariServer decides. Privileged ctl verbs are queued with the requesting operator's name, and the active server replays them.
- **PHP never signs.** Certificates are signed by solariServer (agents) or by step-ca by hand (hosts and services).
- **No private key leaves the host that made it.** Enrollment moves CSRs and certificates only.
- **Secrets live in files referenced by path**: the php-fpm pool configuration, unit environment files, `/etc/solarinet/`, `/etc/solari/`. They are never written to git, logs or the SoR.
- **License keys are never stored.** Inventory keeps a reference to where a license lives.
- **Destructive verbs name an operator** and, for decommissioning, need a confirmation token.

Known gaps

GAP	WHERE IT IS TRACKED
All SolariNet daemons on xenon run as `User=jason`, not the dedicated `solari` account	F-04
The server checks the certificate CN but cannot compare the stored leaf fingerprint on live nng connections	Code comment; nng API limitation
CRL generation, publication and activation timers are not installed on xenon	Observed 2026-11-03
Unsigned provisioning is blocked unless overridden	F-07
CSP needs `unsafe-eval` for the in-browser JSX compiler	F-31
Backups are accepted when non-empty; no restore drill has been proven	F-27
Fleet mTLS is configurable rather than enforced; some transport tests are plaintext	F-29

The F-numbers refer to `docs/SolariNet_Findings_and_Recommendations_2026-08-09.md`.

CHAPTER 14

Build, test and CI

Building and testing locally

```
# C tier: full server build with transport
git clone --recursive ssh://forgejo@cesium.akoria.net:2222/jason/Solarinet.git
cmake -S . -B build -DSOLARI_BUILD_SERVER=ON -DSOLARI_WITH_IO=ON \
      -DSOLARI_WITH_SQLITE=ON -DSOLARI_WITH_JSON=ON -DSOLARI_WITH_DISCOVERY_TOOLS=ON
cmake --build build -j
ctest --test-dir build --output-on-failure

# Panel daemon and firmware host tests
make -C status-panel/daemon
```

PHP needs no build: route and policy tests run with the system PHP from `tests/dashboard`. Python components have their tests beside them (`deploy/sorsync/test_*.py`, `deploy/notify/test_*.py`, `deploy/dns-release/test_*.py`) and run with `pytest`.

DIRECTORY	CONTAINS
`tests/unit`	Unity C tests, 30 `add_test` entries
`tests/fuzz`	libFuzzer harnesses for frame and TLV parsing
`tests/fault`	Fault-injection test for the spool
`tests/integration`	End-to-end tests including enrollment over real HTTPS, PHP, ctl, MariaDB and CA
`tests/dashboard`, `tests/browser`	PHP route, policy and manifest tests; Playwright browser tests
`tests/deploy`, `tests/netdb`	Release, unit-file and zone-generation tests

Continuous integration

CI runs on Forgejo Actions. `.forgejo/workflows/ci.yml` is generated from the GitHub workflow by `tools/ci/render_forgejo_workflow.py`; edit the source and regenerate rather than editing the Forgejo file. Jobs run on the `solarinet-ci-host` runner: an ephemeral libvirt VM on benzene, on 192.168.123.0/24, firewalled from the rest of the network.

JOB	COVERS
python-integrations	Python daemons and helpers
acme-integration	ACME and certificate tooling
sorsync	Emitter and appliers against a RabbitMQ matrix and MariaDB 11; SoR migration rehearsals
protocol-fuzz	Short fuzzing runs on the SCP parsers
core	Library and agent build and unit tests
server	Server with ASAN and UBSAN, MariaDB, Playwright, release binary build and release tests
rackwire-browser	RackWire embed in a browser
dashboard	PHP routes, policy, manifest, CSRF and SQL guards
panel	Panel daemon and firmware host tests
cross	arm64, arm32 and Windows cross builds

The GitHub mirror runs the same set and adds macOS. Whether the latest `main` was green on 2026-11-03 was not checked. UNVERIFIED

CHAPTER 15

Release and deploy

From commit to xenon

xenon runs only code from immutable releases. Nothing in production executes from a working checkout, and a verifier fails the release if anything does.

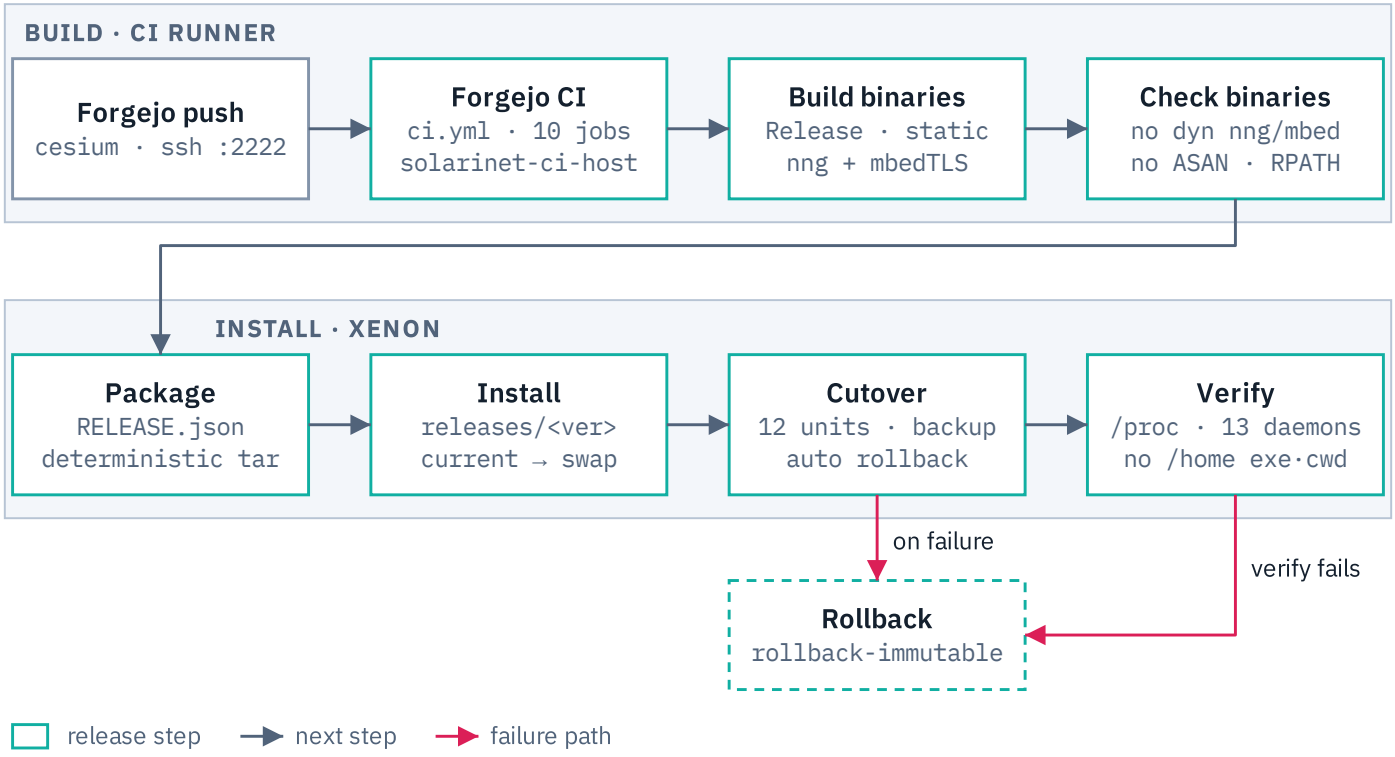


Figure 14 Release pipeline from push to verified immutable cutover.

STEP	TOOL	WHAT IT GUARANTEES
Build	`deploy/build-release-binaries.sh`	Release build with system dependencies, server, transport, discovery, SQLite and JSON on, tests off
Check	`deploy/dashboard/check-release-binaries.py`	No dynamic nng, mbedTLS or sanitizer libraries; no RPATH
Package	`deploy/dashboard/package-release.py`	Requires the `SOLARI-TRANSPORT:nng+mbedtls` marker and a matching RackWire pin; writes `RELEASE.json` with a SHA-256 per file; deterministic tar
Install	`deploy/dashboard/install-release.py`	Unpacks to `/opt/solarinet/releases/<version>`, builds the Python `.venv`, swaps `/opt/solarinet/current` atomically. Restarts nothing.
Cut over	`deploy/host-configs/xenon/cutover-immutable.sh`	Installs 12 unit files and starts or restarts the units that were in service, backing up the old unit files to `/var/backups/solarinet-immutable-<UTC>-<pid>`; rolls back automatically on failure
Verify	`verify-immutable-processes.py`	Scans `/proc` for 13 recognized daemon names; fails if a recognized daemon's executable or working directory, or a code-bearing argument (a `.py` file, a daemon binary or a `/build-io/` path), is under the development checkout
Roll back	`rollback-immutable.sh`	Restores the backed-up units and previous release

The cut-over set is `sor-emit`, `sor-apply-dns`, `sor-apply-pihole`, `sor-reconcile`, `notify`, `alertbridge`, `unifipolld`, `solarinet-client`, `solarinet-monitor`, `solarinet-snmp`, `solari-mdns-inspect` and `solarinet-server`. The release contract is in `deploy/dashboard/RELEASE-CONTRACT.md`; `deploy/release/test_immutable_release.py` tests it.

Versions and order of operations

Releases are named `rc-YYYYMMDD-<short sha>-io`; the `-io` suffix marks a build with the nng and mbedTLS transport. Releases are cut from `main`. Git work lands on Forgejo first; GitHub is a mirror.

1. Merge to `main` on Forgejo; wait for CI.
2. Apply database migrations with `tools/migrate.py`, SoR and monitoring separately, before the release that needs them. Migrations are not part of the release tarball's cut-over.
3. Build, package, install and cut over.
4. Run the verifier; check the dashboard and the alert path.
5. If service-worker-cached files changed shape, the release must also bump `CACHE_VERSION` in `sw.js`.

> **WHO DEPLOYS**
>
> Deployment to xenon and other hosts is done by the lead on the project, not by contributors or agents working on a branch. Do not run cut-over, migration or DNS release commands against live hosts from a feature branch.

CHAPTER 16

Repository map, standards and cautions

Repository map

PATH	CONTENTS
`CMakeLists.txt`, `Makefile`, `cmake/`	C build, wrappers and cross toolchains
`include/`, `lib/solari/`	Public headers and the shared C library
`src/client`, `src/monitor`, `src/server`	The agents and the server with its tools
`dashboard/api`	PHP API: `index.php`, `lib/`, `routes/`
`dashboard/public`	Static UI, vendored React and Babel, fonts, service worker, RackWire submodule
`db/`	Monitoring DB migrations and maintenance SQL
`netdb/`	Zone generator, host list, SoR migrations, rollback and schema snapshots
`deploy/`	Everything that runs or installs outside the C tier: sorsync, alertbridge, notify, unifi, fleet, dns-release, dns-rpz, crl, pki, enrollment, systemd units, host configs, release scripts
`status-panel/`	Panel daemon, firmware, shared protocol, contracts and work-package records
`firmware/tab5`	Firmware for the optional Tab5 push-approval device (PlatformIO)
`integrations/`	`mcp`: a parked Model Context Protocol connector
`tests/`	All test suites (Chapter 14)
`tools/`	`migrate.py`, `export_schema.py`, `audit_schema.py`, CI and documentation tooling
`docs/`	Plans, remits, findings, manuals and the authorization manifest
`third_party/`	Vendored nng, mbedTLS, SQLite, cJSON and Unity

One-off reports and incident files at the repository root are records, not code.

Coding standards

- **C**: C99, warning-clean under `-Wall -Wextra -Wpedantic`; camelCase; opaque handles instead of exposed structures; every function documents purpose, input and output; no allocation on hot paths without a bound; parsers fuzzed.
- **PHP**: PHP 8 with `strict_types`; functional style; prepared statements only, with no named placeholder reused within one statement (a CI test checks this); every write in a transaction with `FOR UPDATE` on the rows it changes. Newer route files use snake_case helper functions; follow the style of the file you are in.
- **JavaScript**: browser only, JSX compiled in the browser, no build tooling. **Python**: standard library where possible; daemons log, back off and reconnect on transient database or broker errors rather than crash.
- **SQL**: new migrations only, never edits to applied ones; provenance on every asserted row; native `INET6` for addresses.

Operational cautions

CAUTION	WHY
Alarm tests are loud	They reach the panel, phones and email. Warn the household first.
Destructive ctl verbs	They need `op=`; decommission needs a confirm token. Never script around the confirm step.
DNS changes go live in seconds	Any write to a CDC table re-renders zones. dns-release will roll back a failed verification, but a valid wrong zone is served.
Two databases, two migration trees	Monitoring and SoR migration numbers are independent. Name the database every time.
INET6	Use `CAST`; `INET6_NTOA` and bare `INET6_ATON` fail silently on the native type.
Keycloak bind account	If its AD password expires, every dashboard login fails.
Health means the application answers	A port that accepts connections is not healthy until a real request succeeds.
Backups are unproven	No restore drill has been done (F-27). Do not rely on them for risky schema work: take your own dump.
Do not commit inside the RackWire submodule	Change RackWire in its own repository and bump the pin.
Service worker version	Forgetting to bump it leaves iPads on stale code.

CHAPTER 17

Interrelationship matrix

Component matrix

Each row starts a connection to, or acts on, the column. Codes: **H** HTTPS · **C** ctl socket · **N** nng over mutual TLS · **S** SQL to MariaDB · **A** AMQP · **D** DNS (query, NOTIFY or transfer) · **F** local zone files and reload · **X** process execution or SSH. Diagonal entries are traffic between instances of the same component.

Table 17.1 · Who initiates to whom (row → column)

	BROWSER	DASHBOARD	SOLARISERVER	AGENTS	MON. DB	SOR	RABBITMQ	SOR SYNC	ALERTS	BIND	KEYCLOAK	STEP-CA	PROVISIONING	SOLARIPANEL	UNIFIPOLLD
Browser		H									H				
Dashboard			C		S	S				D	H	H			
solariServer				N	S								X		
Agents			N	N											
Monitoring DB															
SoR															
RabbitMQ															
SoR sync					S	S	A			F	X				
Alert + notify					S		A								
BIND										D					
Keycloak · AD															
step-ca															
Provisioning				X		S									
solariPanel		H													
unifipolld						S									

Connections in detail

FROM	TO	PROTOCOL AND PORT	DIRECTION	AUTHENTICATION
Browser	Dashboard	HTTPS :443	request/response; UI polls every 10 s	OIDC session cookie; CSRF on writes
Browser	Keycloak	HTTPS :8443	redirect for sign-in	AD credentials or passkey (Keycloak policy)
Dashboard	Keycloak	HTTPS :8443	code exchange, JWKS fetch	Client secret from pool environment
Dashboard	solariServer	Unix socket `/run/solari/solariCtl.sock`	one request per connection	File mode + `SO_PEERCRED`; privileged verbs queued
Dashboard	Monitoring DB	MariaDB 127.0.0.1:3306	read/write	Per-user grant; password in pool environment
Dashboard	SoR	MariaDB 10.1.0.200:3306	read/write	`solari_dashboard` grant
Dashboard	step-ca	HTTPS chlorine:443	read only	Server certificate checked against `STEPCA_CA_FILE`
Dashboard	BIND	DNS :53	queries for DNS views	None
Agents	solariServer	nng :7701 PUSH, :7702 REQ	agent → server; ACK on :7702	Mutual TLS, SolariNet CA
solariServer	Agents	nng :7703 PUB (agents connect)	server → agents	Mutual TLS, SolariNet CA
Monitors	Monitors	nng PUSH/PULL gossip	peer to peer	Mutual TLS; CN `monitor.<fqdn>`
solariServer	Monitoring DB	MariaDB 127.0.0.1:3306	read/write; lease row	User `solari`; password from `SOLARI_DB_PASS`
solariServer	Provisioning helpers	fork + execve	parent → child	Runs as the server's user; no shell
Provisioning helpers	SoR	MariaDB :3306	read; `prov_runs` writes	Option file `/etc/solarinet/prov-sor.cnf`
Provisioning helpers	benzene → fleet hosts	SSH, then Ansible over SSH	xenon → benzene → target	SSH keys, BatchMode
sor_emitd	SoR → RabbitMQ	MariaDB; AMQP :5672	outbox → `sor.events`	Per-daemon grant; AMQP user, vhost `solari`
Appliers	RabbitMQ	AMQP :5672	consume `sor.apply.*`	AMQP user
sor_apply_dnsd	BIND (xenon)	Local files + reload	render → release	Local; dns-release refuses writable config
sor_apply_ad	Samba AD (radium)	`samba-tool`, local	apply	Runs as root on radium **UNVERIFIED**
sor_reconcile_discovery	Monitoring DB → SoR	MariaDB	observed → SoR	Per-daemon grants

FROM	TO	PROTOCOL AND PORT	DIRECTION	AUTHENTICATION
alertbridge	Monitoring DB → RabbitMQ	MariaDB; AMQP	`alertEvent` → `notify.events`	DB user; AMQP user
notifyd	RabbitMQ → channels	AMQP; SSH, SMTP, MQTT, Web Push	consume and fan out	Per-sender credentials in `notify.conf`
unifipolld	UniFi controller → SoR	HTTPS; MariaDB	every 15 s	Controller account; per-daemon grant
solariPanel	Dashboard	HTTPS :443	poll every 5 s; post state	`panel` viewer session
solariPanel	Panel firmware	USB CDC serial	both ways	Physical connection; CRC-checked frames
BIND primary	Secondaries	DNS NOTIFY, AXFR/IXFR	xenon → radium, chlorine, steel	Transfer allow-lists by address

APPENDIX A

Ports

Ports

HOST	PORT	SERVICE
xenon	443 · 9443	Dashboard (Apache); legacy vhost
xenon	7701 · 7702 · 7703	solariServer PULL · REP · PUB (mutual TLS)
xenon	3306 (local)	Monitoring DB
xenon	53	BIND primary for `akoria.net`, RPZ
xenon	unix	`/run/solari/solariCtl.sock`, `/run/php/solarinet85-fpm.sock`
cesium	3306 · 3000 · 2222	SoR · Forgejo HTTP · Forgejo SSH
benzene	5672 · 15672 · 1883	AMQP · RabbitMQ management · MQTT
benzene	8080 · 69	Netboot HTTP · TFTP
benzene	3306	SoR replica
radium	88 · 389 · 636 · 53	Kerberos · LDAP · LDAPS · AD DNS
radium	8443	Keycloak
chlorine	443 · 53	step-ca · BIND secondary
steel	53	BIND secondary
lithium	7443	solariPanel Wi-Fi listener (not in use)

APPENDIX B

Discrepancy register

Discrepancy register

Where a document in the repository disagrees with the code or the live system, this manual follows the code or the live system. Each row names the document so it can be corrected.

DOCUMENT OR FILE	SAYS	ACTUALLY
`docs/SolariNet_Developer_Reference.html`	No migration ledger in production	SoR ledger records 024–037 **OBSERVED**
`docs/SolariNet_Developer_Reference.html`	Frozen capabilities	The freeze was rescinded on 2026-10-30; cautions such as manual remote CA still apply
`docs/SolariNet_Developer_Reference.html`	DNS apply has no rollback	dns-release rolls back failed generations
`deploy/dns-release/README.md`	Not installed on the live DNS services	Live on xenon; `named.conf.local` includes the managed release **OBSERVED**
`deploy/notify/README.md`	Older sender list	Registry in `senders/__init__.py`
`server.conf.ctl-fragment`	Legacy mode treats every peer as operator	Non-operator peers are classed DASHBOARD
Stock client and monitor units in `deploy/systemd`	—	Omit `--loop` **INFERRED**
`monitor.conf`	`gossipIntervalSec` sets gossip rate	Parsed but unused
`netdb/sor/SCHEMA.md`	Schema diagram	Stale against migrations 033–037
`status-panel/README.md`	Daemon `solari-paneld`	Binary `solariPanel`, unit `solari-panel.service`
Unit templates for `solariScanWorker`	Scan worker deployed	Not installed on xenon **OBSERVED**
`deploy/crl`	CRL timers	Not installed on xenon **OBSERVED**
Repository service accounts	Daemons run as `solari`	xenon units run as `jason` **OBSERVED**
Inventory I1 plan	Migration numbers	Shipped as SoR 033–037

APPENDIX C

Glossary

Glossary

Active server	The solariServer holding the lease; the only one that ingests and runs the ctl queue.
Applier	A daemon that consumes SoR change events and changes an external system.
CDC	Change data capture: triggers write each change to `sor_outbox`.
ctl	The Unix-socket control protocol into solariServer.
Disposition	Whether an alert event is published or suppressed.
Durable report	An agent report sent on :7702 and held in the spool until acknowledged.
Entity	A thing on the network in the SoR: host, appliance, service or device.
Generation	One installed set of zone files under dns-release.
HRW	Highest-random-weight (rendezvous) hashing, used to share probe targets.
Lease	The database row that decides which server is active.
Live flag	Generated column that makes uniqueness ignore soft-deleted rows.
Outbox	`sor_outbox`, the CDC queue table in the SoR.
Provenance	Source, kind and time recorded with each asserted SoR row.
SCP	SolariNet Control Protocol, the framed binary protocol between agents and server.
SoR	System of record: the `sor` database on cesium.
Spool	The SQLite store of undelivered agent reports.
Tombstone	A retired row kept for history, hidden from live views.